

BOMBEROS DE TODOS SANTOS

Documento Maestro del Proyecto Web

Planeación · Arquitectura · Diseño · Implementación

Sprint intensivo de 7 días | Claude Code + Codex

Versión 1.0 · Mayo 2026

Estado: Planeación cerrada, listo para implementación

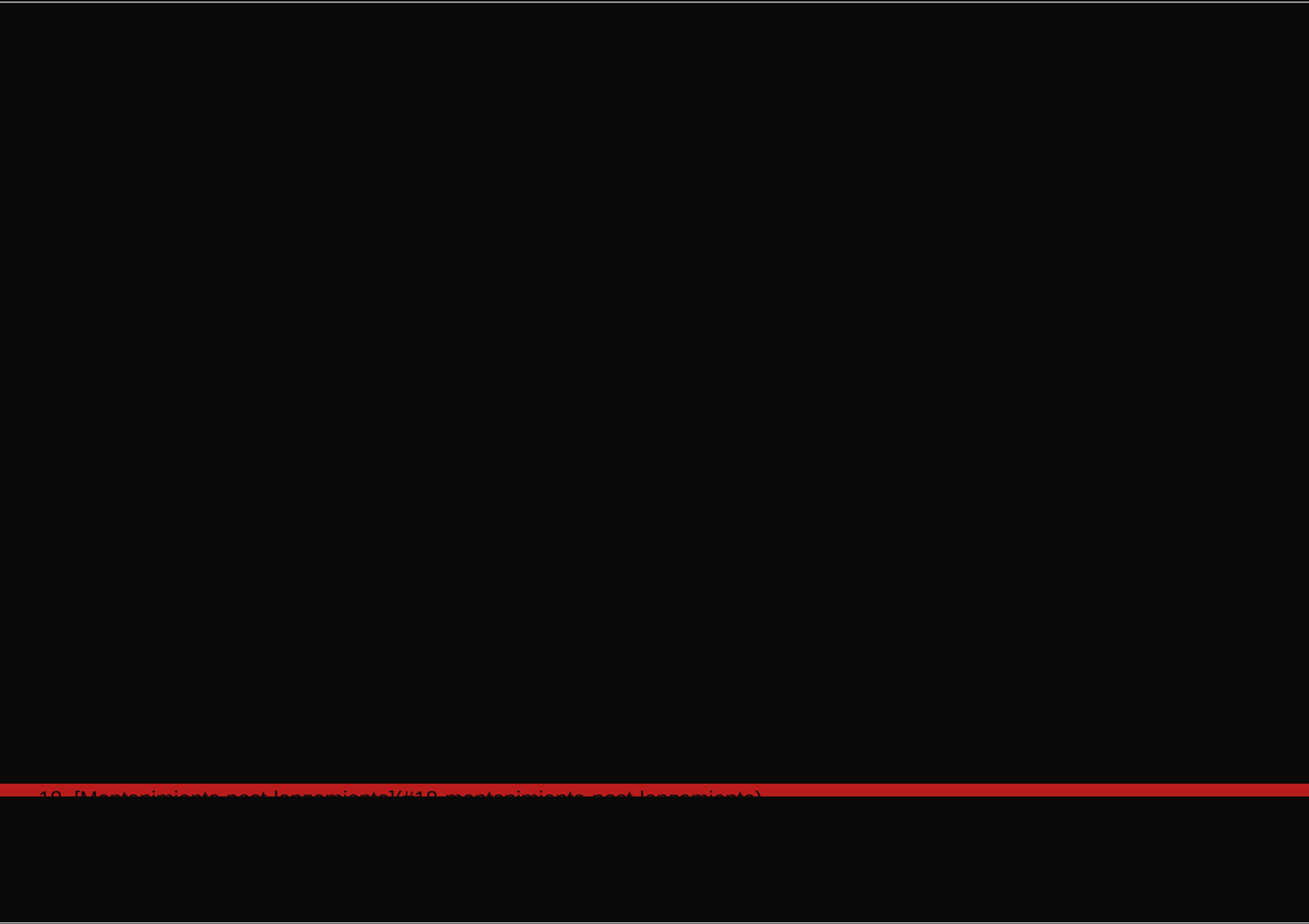
Versión: 1.0

Fecha: Mayo 2026

Estado: Planeación cerrada, listo para implementación

Timeline: Sprint intensivo de 1 semana

Equipo: 1 humano + Claude Code + Codex



1.9. [Mantenimiento post-lanzamiento]/(1.9. mantenimiento post-lanzamiento)

Métricas de éxito del MVP

Métrica	Objetivo
Tiempo de carga (Lighthouse Performance)	≥90
Accesibilidad (Lighthouse)	≥95
Páginas indexables	100% del contenido público

Métrica	Objetivo
Tiempo del admin para subir una necesidad	<3 minutos
Cobertura bilingüe	100% UI, 100% contenido inicial
Tasa de errores en producción (Sentry)	<0.5%

Entregables principales

1. Sitio web público bilingüe con wishlist de equipo, transparencia, eventos, voluntariado y noticias.
2. Panel de administración interno para gestión total del contenido.
3. Pipeline de CI/CD automático con Vercel.
4. Documentación de operación para el bombero administrador.
5. Plan de mantenimiento mensual.

2. Contexto del proyecto

2.1 Sobre la organización

Los Bomberos Voluntarios de Todos Santos son una pequeña central que opera con bomberos voluntarios y subsiste principalmente gracias a donaciones extranjeras de equipo, cursos de capacitación y eventos comunitarios de recaudación. Operan en un contexto particular:

- **Recursos limitados:** dependen de donaciones, no de presupuesto público estable.
- **Voluntariado activo:** muchos voluntarios, rotación constante.
- **Audiencia mixta:** donantes extranjeros (principalmente EE.UU. y Canadá) + comunidad local mexicana.
- **Figura legal por confirmar:** afecta el manejo de pagos (Stripe vs PayPal vs transferencia directa).

2.2 Problema que resuelve el sitio

Sin un sitio web propio, la central enfrenta varias limitaciones:

- **Comunicación dispersa:** las necesidades se comunican por Facebook, WhatsApp y boca a boca, sin un registro central.
- **Falta de transparencia:** los donantes no pueden ver qué se hizo con donaciones pasadas, lo cual reduce confianza y donaciones recurrentes.
- **Sin canalización clara:** quien quiere ayudar no sabe específicamente qué se necesita ni cuánto cuesta.
- **Eventos invisibles:** los eventos de recaudación tienen alcance limitado al círculo local.

El sitio resuelve esto centralizando: necesidades, donaciones, eventos, voluntariado e historia de la central.

2.3 Filosofía del proyecto

Tres principios guían cada decisión:

- 1. **La transparencia genera confianza.** Toda donación, todo equipo recibido, toda métrica relevante debe poder mostrarse públicamente. Esto es lo que diferencia a esta central de un sitio genérico de ONG.
- 2. **El contenido vive más que el código.** El sitio fracasa si el admin no puede mantenerlo. Cada decisión técnica privilegia la facilidad de administración sobre la elegancia técnica.
- 3. **MVP enfocado, no MVP recortado.** Mejor tres páginas excelentes que diez mediocres. El alcance de la semana 1 prioriza lo que mueve la aguja: necesidades, donaciones y transparencia.

3. Decisiones de planeación

3.1 Audiencia

Segmento	Idioma	Necesidad principal
Donantes extranjeros (EE.UU., Canadá)	Inglés	Saber exactamente qué financian, recibir comprobante
Comunidad local (Todos Santos, BCS)	Español	Conocer eventos, voluntariado, transparencia
Voluntarios potenciales	Ambos	Saber cómo aplicar, qué se espera

Implicación: sitio bilingüe completo con i18n nativa, no traducción automática.

3.2 Priorización de objetivos

Ordenados de mayor a menor prioridad:

- 1. **Solicitar equipo específico** (corazón del sitio — wishlist con estado y progreso)
- 2. **Recibir donaciones monetarias** (página /donar con métodos múltiples)
- 3. **Transparencia y comunicación** (muro de donaciones, historia, métricas)
- 4. **Promocionar eventos y cursos** (calendario público)

Implicación clave: la home no es un "botón donar gigante" — es una **vitrina de necesidades específicas** con costo, estado y urgencia.

3.3 Métodos de pago

Método	MVP	Post-MVP
Stripe	Preparado, desactivado	Activo cuando se confirme figura legal
PayPal	Activo	Continúa
Transferencia bancaria	Activo	Continúa
Mercado Pago	No	Evaluar si crece base local

Razón: Stripe México requiere RFC y cuenta bancaria empresarial. Activarlo sin la figura legal correcta puede causar congelamiento de cuenta.

3.4 Transparencia de donaciones

Decisión: registro público de donaciones con opt-in del donante.

- Se muestra: nombre del donante (si aceptó), país, equipo financiado, fecha aproximada.
- No se muestra: email, monto exacto (opcional), método de pago.
- Solo se muestran donaciones **verificadas por el admin** (no auto-publicadas).

3.5 Voluntariado

Decisión: formulario público con flujo de aprobación admin.

- Formulario captura: nombre, contacto, edad, ciudad, habilidades, disponibilidad, motivación.
- Estado por defecto: pending.
- Admin recibe email de notificación y aprueba/rechaza desde el panel.
- Voluntario recibe email automático con la decisión.

3.6 Identidad visual

Aspecto	Decisión
Tono	Heroico y emotivo
Color primario	Rojo bombero #B91C1C
Color secundario	Negro profundo #0A0A0A
Fondo neutral	Hueso #F5F5F4
Tipografía display	Oswald (headlines)
Tipografía cuerpo	Inter (UI, párrafos)

Aspecto	Decisión
Fotografía	Real, no stock; documental

3.7 Timeline

Sprint intensivo de 7 días con dos agentes de código trabajando en paralelo (Claude Code + Codex).
Detalles en [sección 12](#12-plan-de-sprint-7-días).

4. Arquitectura de información

4.1 Sitemap completo

```
/ Home (hero + 3 necesidades destacadas + stats + CTA)
/necesidades Lista filtrable de equipment_needs
■■■ /[slug] Detalle con fotos, specs, progreso, donantes
/donar Métodos de donación + form "voy a donar"
/transparencia Muro de donantes verificados + reporte
/eventos Lista de eventos próximos y pasados
■■■ /[slug] Detalle del evento
/cursos Cursos disponibles / impartidos
/voluntarios Info + formulario de aplicación
/nosotros Historia, equipo, ubicación, contacto
/noticias Blog: noticias, incidentes, agradecimientos
■■■ /[slug] Detalle de noticia

/admin Dashboard (login required)
/admin/login Magic link
/admin/necesidades CRUD equipment_needs
/admin/donaciones Verificar y aprobar
/admin/eventos CRUD events
/admin/voluntarios Aprobar/rechazar
/admin/noticias CRUD posts
/admin/stats Editar station_stats anual
/admin/settings Editar site_settings
```

```
/api/webhooks/stripe (POST) Webhook de Stripe  
/api/webhooks/paypal (POST) Webhook de PayPal  
/api/og/[type] (GET) Open Graph images dinámicas  
/sitemap.xml Auto-generado  
/robots.txt Auto-generado
```

4.2 Rutas con localización

Estructura del App Router:

```
src/app/  
■■■ [locale]/ ← rutas públicas en /es/* y /en/*  
■ ■■■ ...  
■■■ admin/ ← admin solo en español, no localizado  
■■■ api/ ← endpoints técnicos, no localizados
```

Configuración de middleware: redirige / → /es/ (o /en/ según Accept-Language).

4.3 User flows críticos

Flow 1 — Donante extranjero quiere financiar equipo específico:

```
Home (EN, detecta idioma del browser)  
→ Ve "Equipment that saves lives" + 3 needs destacadas  
→ Click en "SCBA breathing apparatus" ($3,200 USD, 0/2 covered)  
→ Lee detalle: specs, fotos, urgencia  
→ Click "Donate this equipment"  
→ Página /donate con need pre-seleccionada  
→ Elige método (PayPal en MVP, Stripe en v1.1)  
→ Completa donación  
→ Email de gracias automático  
→ Opcionalmente aparece en /transparency
```

Flow 2 — Local quiere ser voluntario:

```
Home (ES)  
→ Click "Voluntarios" en nav  
→ Lee requisitos, compromisos, beneficios
```

- Llena formulario (10 campos)
- Submit → ve mensaje "Recibimos tu aplicación"
- Email automático "Te contactaremos en 5 días hábiles"
- Admin recibe email de notificación
- Admin revisa en /admin/voluntarios
- Aprueba o rechaza con nota
- Voluntario recibe email con decisión

Flow 3 — Admin sube nueva necesidad:

- /admin/login → magic link al email
- Click link → autenticado
- /admin/necesidades → "Nueva"
- Llena form (bilingüe en mismo screen):
 - Título ES + EN
 - Descripción ES + EN
 - Categoría, prioridad
 - Costo estimado USD + MXN
 - Cantidad necesaria
 - Subir fotos (drag & drop)
 - Especificaciones técnicas
- Toggle "Publicada" + "Destacada en home"
- Guardar
- Aparece inmediatamente en sitio público

Flow 4 — Admin verifica donación recibida (en especie o bancaria):

- Recibe notificación por email de form "voy a donar"
- /admin/donaciones → ve la entry con status "pending"
- Verifica con su estado de cuenta o recepción física
- Edita la entry: marca is_verified=true
- Si el donante autorizó: marca is_public=true
- Si está vinculada a equipment_need: actualiza quantity_received
- Guarda → aparece en /transparencia
- Opcionalmente: marca el need como "fulfilled" si se completó

4.4 Jerarquía de navegación

Header desktop:

[Logo] Necesidades | Transparencia | Eventos | Nosotros | Más ▼ | ES/EN | [Donar]

■ Voluntarios

Cursos

Noticias

Header móvil: logo + hamburger + botón "Donar" siempre visible.

Footer:

Columna 1: Contacto (dirección, teléfono, email, emergencia)

Columna 2: Redes sociales

Columna 3: Datos bancarios para donaciones directas

Columna 4: Links legales (próximamente)

Línea inferior: © + año + créditos

5. Stack tecnológico

5.1 Decisiones de stack y justificación

Capa	Tecnología	Versión	Por qué
Framework	Next.js	15.x (App Router)	Full-stack, SSR/SSG, RSC, i18n nativa, deploy trivial
Lenguaje	TypeScript	5.x estricto	Type-safety end-to-end, autocompletado, menos bugs
Runtime	Node.js	20 LTS	Compatible con Vercel y Supabase SDK
Package manager	pnpm	9.x	Más rápido que npm, lockfile determinístico, ideal con monorepos futuros
Estilos	Tailwind CSS	3.x	Velocidad de desarrollo, design tokens, sin CSS huérfano
Componentes UI	shadcn/ui	latest	Copy-paste, customizable, accesibilidad incluida

Capa	Tecnología	Versión	Por qué
Base de datos	Supabase (Postgres 15)	latest	BaaS completo, RLS, free tier suficiente
Auth	Supabase Auth	latest	Magic link sin password, sesiones JWT
Storage	Supabase Storage	latest	Buckets públicos/privados, CDN incluido
Forms	react-hook-form + Zod	latest	Validación end-to-end con tipos derivados
i18n	next-intl	3.x	Mejor DX que next-i18next con App Router
Email	Resend	latest	3,000 emails/mes free, DX excelente
Pagos	Stripe + PayPal SDK	latest	Stripe para tarjetas internacionales, PayPal familiar
Hosting	Vercel	latest	Deploy automático, preview deploys, edge network
Analytics	Vercel Analytics + Plausible	latest	Privacy-friendly, sin cookies invasivas
Error tracking	Sentry (free tier)	latest	Catch errores en producción
Imágenes	next/image + Supabase Storage	nativo	Optimización automática, lazy loading, AVIF/WebP
Iconos	lucide-react	latest	Tree-shakeable, consistente con shadcn
Fechas	date-fns	latest	Tree-shakeable, locale-aware (es, en)

5.2 Estimación de costos mensuales

Servicio	Tier	Costo estimado MVP
Vercel	Hobby	\$0 USD
Supabase	Free	\$0 USD (hasta 500MB DB, 1GB storage, 2GB bandwidth)
Resend	Free	\$0 USD (hasta 3,000 emails/mes)
Sentry	Developer	\$0 USD (5K errores/mes)

Servicio	Tier	Costo estimado MVP
Plausible	\$9/mes opcional	\$0-9 USD
Dominio	Anual	~\$15 USD/año
Stripe / PayPal	Por transacción	3-4% + \$0.30 USD por donación

Total operativo: \$0-9 USD/mes + dominio anual + comisiones por donación recibida.

5.3 Dependencias del package.json final

```
{
  "name": "bomberos-todos-santos",
  "version": "0.1.0",
  "private": true,
  "scripts": {
    "dev": "next dev",
    "build": "next build",
    "start": "next start",
    "lint": "next lint",
    "typecheck": "tsc --noEmit",
    "db:types": "supabase gen types typescript --local > src/types/database.ts",
    "db:reset": "supabase db reset",
    "db:diff": "supabase db diff"
  },
  "dependencies": {
    "next": "^15.0.0",
    "react": "^19.0.0",
    "react-dom": "^19.0.0",
    "@supabase/supabase-js": "^2.45.0",
    "@supabase/ssr": "^0.5.0",
    "next-intl": "^3.20.0",
    "zod": "^3.23.0",
    "react-hook-form": "^7.53.0",
    "@hookform/resolvers": "^3.9.0",
    "tailwindcss": "^3.4.0",
    "tailwind-merge": "^2.5.0",
```

```
"clsx": "^2.1.0",
"class-variance-authority": "^0.7.0",
"lucide-react": "^0.450.0",
"@radix-ui/react-dialog": "^1.1.0",
"@radix-ui/react-dropdown-menu": "^2.1.0",
"@radix-ui/react-label": "^2.1.0",
"@radix-ui/react-select": "^2.1.0",
"@radix-ui/react-toast": "^1.2.0",
"@radix-ui/react-slot": "^1.1.0",
"date-fns": "^4.1.0",
"stripe": "^17.0.0",
"@paypal/react-paypal-js": "^8.7.0",
"resend": "^4.0.0",
"@sentry/nextjs": "^8.0.0",
"@vercel/analytics": "^1.3.0"
},
"devDependencies": {
  "typescript": "^5.6.0",
  "@types/node": "^22.0.0",
  "@types/react": "^19.0.0",
  "@types/react-dom": "^19.0.0",
  "eslint": "^9.0.0",
  "eslint-config-next": "^15.0.0",
  "prettier": "^3.3.0",
  "prettier-plugin-tailwindcss": "^0.6.0",
  "supabase": "^1.200.0",
  "autoprefixer": "^10.4.0",
  "postcss": "^8.4.0"
},
"packageManager": "pnpm@9.0.0"
}
```

6. Arquitectura de repositorios

6.1 Decisión: monorepo único de Next.js + Supabase como BaaS

Un solo repo en GitHub. Next.js es full-stack: contiene UI, Server Actions (lógica de servidor) y Route Handlers (webhooks). Supabase es el backend gestionado y no entra al repo — solo sus migraciones SQL versionadas.

Por qué no separar frontend y backend:

1. **Type-safety end-to-end:** los tipos generados de Supabase se usan en formularios, Server Actions y páginas públicas. Con repos separados, se duplicaría trabajo.
2. **Server Actions reemplazan endpoints:** `createEquipmentNeed(formData)` es funcionalmente idéntica a `POST /api/equipment-needs`, pero sin CORS, sin documentar API, sin autenticación entre servicios.
3. **Mantenimiento simplificado:** un `package.json`, un `vercel.json`, un workflow de CI.
4. **Deploy atómico:** push a main despliega todo simultáneamente.

Cuándo sí valdría la pena separar (no aplica hoy):

- Trabajos asíncronos largos (>10s): se resuelve con Supabase Edge Functions, no con repo aparte.
- Múltiples clientes consumiendo la misma API (web + móvil + widget): no es el caso del MVP.
- Equipos grandes con dueños diferentes de front y back: no aplica al sprint.

6.2 Estructura completa del repo

```
bomberos-todos-santos/  
  ■■■ .github/  
  ■ ■■■ workflows/  
  ■ ■ ■■■ ci.yml # Lint + typecheck + build en cada PR  
  ■ ■ ■■■ preview.yml # Preview deploy en cada PR  
  ■ ■■■ pull_request_template.md # Template obligatorio de PRs  
  ■ ■■■ CODEOWNERS # Quien aprueba qué (opcional)  
  ■  
  ■■■ public/  
  ■ ■■■ favicon.ico  
  ■ ■■■ apple-touch-icon.png  
  ■ ■■■ og-default.jpg # Imagen OG fallback  
  ■ ■■■ logo.svg # Logo SVG  
  ■
```

```

    ■■■ src/
  ■ ■■■ app/
    ■ ■■■ [locale]/ # Rutas bilingües ES/EN
    ■ ■ ■■■ layout.tsx # Layout público (Header + Footer)
    ■ ■ ■■■ page.tsx # Home
    ■ ■ ■■■ loading.tsx # Skeleton de carga
    ■ ■ ■■■ error.tsx # Error boundary
    ■ ■ ■■■ not-found.tsx # 404 custom
    ■ ■ ■■
  ■ ■ ■■■ necesidades/
    ■ ■ ■ ■■■ page.tsx # Lista filtrable
    ■ ■ ■ ■■■ [slug]/
    ■ ■ ■ ■■■ page.tsx # Detalle
    ■ ■ ■■
  ■ ■ ■■■ donar/
    ■ ■ ■ ■■■ page.tsx # Métodos de donación
    ■ ■ ■ ■■■ gracias/
    ■ ■ ■ ■■■ page.tsx # Página de agradecimiento
    ■ ■ ■■
  ■ ■ ■■■ transparencia/
    ■ ■ ■ ■■■ page.tsx
    ■ ■ ■■
  ■ ■ ■■■ eventos/
    ■ ■ ■ ■■■ page.tsx
    ■ ■ ■ ■■■ [slug]/page.tsx
    ■ ■ ■■
  ■ ■ ■■■ cursos/
    ■ ■ ■ ■■■ page.tsx # Filtra events por type='course'
    ■ ■ ■■
  ■ ■ ■■■ voluntarios/
    ■ ■ ■ ■■■ page.tsx # Info + form
    ■ ■ ■■
  ■ ■ ■■■ nosotros/
    ■ ■ ■ ■■■ page.tsx
```

```

■ ■ ■ ■
■ ■ ■ ■■ noticias/
■ ■ ■ ■■ page.tsx
■ ■ ■ ■■ [slug]/page.tsx
■ ■ ■
■ ■ ■■ admin/ # NO localizado, solo ES
■ ■ ■ ■■ layout.tsx # Sidebar + auth guard
■ ■ ■ ■■ page.tsx # Dashboard
■ ■ ■ ■■ login/page.tsx # Magic link
■ ■ ■ ■
■ ■ ■ ■■ necesidades/
■ ■ ■ ■ ■■ page.tsx # Lista (DataTable)
■ ■ ■ ■ ■■ nueva/page.tsx # Form crear
■ ■ ■ ■ ■■ [id]/page.tsx # Form editar
■ ■ ■ ■
■ ■ ■ ■■ donaciones/
■ ■ ■ ■ ■■ page.tsx # Lista + filtros
■ ■ ■ ■ ■■ [id]/page.tsx # Detalle + verificar
■ ■ ■ ■
■ ■ ■ ■■ eventos/
■ ■ ■ ■ ■■ page.tsx
■ ■ ■ ■ ■■ nuevo/page.tsx
■ ■ ■ ■ ■■ [id]/page.tsx
■ ■ ■ ■
■ ■ ■ ■■ voluntarios/
■ ■ ■ ■ ■■ page.tsx # Lista pending/approved
■ ■ ■ ■ ■■ [id]/page.tsx # Detalle + aprobar/rechazar
■ ■ ■ ■
■ ■ ■ ■■ noticias/
■ ■ ■ ■ ■■ page.tsx
■ ■ ■ ■ ■■ nueva/page.tsx
■ ■ ■ ■ ■■ [id]/page.tsx
■ ■ ■ ■
■ ■ ■ ■■ stats/page.tsx # Editar station_stats por año

```

```
■ ■ ■ ■■ settings/page.tsx # Editar site_settings singleton
■ ■ ■
■ ■ ■■ api/ # Solo webhooks externos
■ ■ ■ ■■ webhooks/
■ ■ ■ ■■ stripe/route.ts
■ ■ ■ ■■ paypal/route.ts
■ ■ ■ ■■ og/
■ ■ ■ ■■ [type]/route.tsx # OG image generator dinámico
■ ■ ■
■ ■ ■■ globals.css # Tailwind + custom CSS vars
■ ■ ■■ sitemap.ts # Sitemap.xml dinámico
■ ■ ■■ robots.ts # robots.txt dinámico
■ ■
■ ■■ actions/ # Server Actions (el "backend")
■ ■ ■■ auth.ts # signIn, signOut
■ ■ ■■ equipment-needs.ts # createNeed, updateNeed, deleteNeed, toggleFeatured
■ ■ ■■ donations.ts # createDonation, verifyDonation, togglePublic
■ ■ ■■ events.ts
■ ■ ■■ posts.ts
■ ■ ■■ volunteers.ts # submitApplication, approveVolunteer, rejectVolunteer
■ ■ ■■ stats.ts
■ ■ ■■ settings.ts
■ ■ ■■ upload.ts # uploadImage, deleteImage (Storage)
■ ■
■ ■■ components/
■ ■ ■■ ui/ # shadcn/ui primitives
■ ■ ■ ■■ button.tsx
■ ■ ■ ■■ input.tsx
■ ■ ■ ■■ label.tsx
■ ■ ■ ■■ textarea.tsx
■ ■ ■ ■■ select.tsx
■ ■ ■ ■■ dialog.tsx
■ ■ ■ ■■ dropdown-menu.tsx
■ ■ ■ ■■ toast.tsx
```



```
■ ■ ■ ■■■ badge.tsx

■ ■ ■ ■■■ ...

■ ■ ■

■ ■ ■■■ layout/

■ ■ ■ ■■■ PublicHeader.tsx # Nav + LangToggle + Donate CTA

■ ■ ■ ■■■ PublicFooter.tsx

■ ■ ■ ■■■ LangToggle.tsx

■ ■ ■ ■■■ AdminSidebar.tsx

■ ■ ■ ■■■ MobileMenu.tsx

■ ■ ■

■ ■ ■■■ public/ # Componentes de páginas públicas

■ ■ ■ ■■■ HeroSection.tsx

■ ■ ■ ■■■ StatsBar.tsx

■ ■ ■ ■■■ NeedCard.tsx

■ ■ ■ ■■■ NeedDetail.tsx

■ ■ ■ ■■■ NeedsFilter.tsx

■ ■ ■ ■■■ DonorWall.tsx

■ ■ ■ ■■■ EventCard.tsx

■ ■ ■ ■■■ PostCard.tsx

■ ■ ■ ■■■ ProgressBar.tsx

■ ■ ■

■ ■ ■■■ admin/ # Componentes del admin

■ ■ ■ ■■■ DataTable.tsx # Reutilizable

■ ■ ■ ■■■ BilingualTextField.tsx

■ ■ ■ ■■■ ImageUploader.tsx

■ ■ ■ ■■■ StatusBadge.tsx

■ ■ ■ ■■■ NeedForm.tsx

■ ■ ■ ■■■ EventForm.tsx

■ ■ ■ ■■■ PostForm.tsx

■ ■ ■ ■■■ VolunteerReviewCard.tsx

■ ■ ■ ■■■ DonationVerifyCard.tsx

■ ■ ■

■ ■ ■■■ forms/

■ ■ ■■■ VolunteerForm.tsx
```

```
■ ■ ■ ■ ■ DonationIntentForm.tsx

■ ■ ■ ■ ■ ContactForm.tsx

■ ■

■ ■ ■ ■ ■ lib/

■ ■ ■ ■ ■ supabase/

■ ■ ■ ■ ■ client.ts # Browser client

■ ■ ■ ■ ■ server.ts # Server Components / Server Actions

■ ■ ■ ■ ■ admin.ts # Service role (server-only)

■ ■ ■ ■ ■ middleware.ts # Refresh de sesión

■ ■ ■

■ ■ ■ ■ ■ stripe.ts # Stripe SDK config

■ ■ ■ ■ ■ paypal.ts # PayPal config

■ ■ ■ ■ ■ resend.ts # Email transaccional

■ ■ ■

■ ■ ■ ■ ■ i18n/

■ ■ ■ ■ ■ config.ts # locales soportados

■ ■ ■ ■ ■ request.ts # next-intl getRequestConfig

■ ■ ■ ■ ■ routing.ts

■ ■ ■ ■ ■ es.json

■ ■ ■ ■ ■ en.json

■ ■ ■

■ ■ ■ ■ ■ validations/ # Zod schemas

■ ■ ■ ■ ■ equipment-need.ts

■ ■ ■ ■ ■ donation.ts

■ ■ ■ ■ ■ event.ts

■ ■ ■ ■ ■ post.ts

■ ■ ■ ■ ■ volunteer.ts

■ ■ ■ ■ ■ settings.ts

■ ■ ■

■ ■ ■ ■ ■ email/ # Templates de email

■ ■ ■ ■ ■ volunteer-received.tsx

■ ■ ■ ■ ■ volunteer-decision.tsx

■ ■ ■ ■ ■ donation-intent.tsx

■ ■ ■ ■ ■ admin-notification.tsx
```

```
■ ■ ■
■ ■ ■■ utils.ts # cn, formatCurrency, formatDate, slugify
■ ■ ■■ constants.ts # Enums replicados, categorías, etc.
■ ■ ■■ seo.ts # generateMetadata helpers
■ ■
■ ■■ types/
■ ■ ■■ database.ts # Auto-generado de Supabase
■ ■ ■■ index.ts # Tipos derivados, props compartidas
■ ■
■ ■■ middleware.ts # i18n locale + auth guard /admin/*
■
■■ supabase/
■ ■■ migrations/
■ ■ ■■ 00001_initial_schema.sql
■ ■ ■■ ...
■ ■■ seed.sql # Datos demo para desarrollo
■ ■■ config.toml # Config local de Supabase CLI
■ ■■ functions/ # Edge functions (post-MVP)
■
■■ docs/
■ ■■ README.md # Setup local, deploy, troubleshooting
■ ■■ ADMIN_GUIDE.md # Manual para el bombero admin
■ ■■ DEPLOYMENT.md # Cómo desplegar a producción
■ ■■ DECISIONS.md # Architecture Decision Records
■ ■■ WORKFLOW.md # Convenciones de Git y código
■
■■ .env.example # Plantilla, sin secretos
■■ .env.local # Local, en .gitignore
■■ .gitignore
■■ .eslintrc.json
■■ .prettierrc
■■ next.config.mjs
■■ tailwind.config.ts
■■ tsconfig.json
```

```

■■■ postcss.config.mjs
■■■ components.json # shadcn/ui config
■■■ package.json
■■■ pnpm-lock.yaml

```

6.3 Convención de nombres

Tipo	Convención	Ejemplo
Carpetas	kebab-case	equipment-needs/
Archivos de componente	PascalCase.tsx	NeedCard.tsx
Archivos de utilidades	kebab-case.ts	format-currency.ts
Server Actions (archivo)	kebab-case.ts	equipment-needs.ts
Server Actions (función)	camelCase	createEquipmentNeed
Tipos TypeScript	PascalCase	EquipmentNeed
Variables de entorno	SCREAMING_SNAKE_CASE	SUPABASE_SERVICE_ROLE_KEY
DB tablas	snake_case_plural	equipment_needs
DB columnas	snake_case	estimated_cost_usd
DB enums	snake_case	priority_level
Branch de Git	kebab-case con prefijo	feat/equipment-needs

7. Modelo de datos

7.1 Diagrama lógico

[illegible]

[illegible]

[illegible]

7.2 Schema SQL completo

El schema completo está en el archivo `supabase/migrations/00001_initial_schema.sql` y se documenta en detalle en la sección [20.1 Apéndice A](#201-apéndice-a-schema-sql-completo). Incluye:

- 8 tablas principales con relaciones explícitas
- 6 enums tipados
- 1 vista materializable para progreso de necesidades
- 1 función helper `is_admin()` con `security definer`
- 18 políticas RLS (Row Level Security)
- 12 índices estratégicos para consultas frecuentes
- 8 triggers para actualizar `updated_at` automáticamente

7.3 Estrategia de Row Level Security

Principio: ninguna tabla queda sin RLS. La seguridad vive en la base de datos, no en el código del frontend.

Tabla	Lectura pública	Escritura pública	Lectura admin	Escritura admin
equipment_needs	Si published=true	No	Total	Total
donations	Si is_public AND is_verified	INSERT (intent)	Total	Total
events	Si published=true	No	Total	Total
posts	Si published=true	No	Total	Total

Tabla	Lectura pública	Escritura pública	Lectura admin	Escritura admin
volunteers	No	INSERT (status='pending')	Total	Total
station_stats	Total	No	Total	Total
site_settings	Total	No	Total	Total
admin_users	No	No	Solo admin	Solo admin

Implicación práctica: un usuario público puede:

- Leer necesidades publicadas
- Leer eventos publicados
- Leer noticias publicadas
- Leer donaciones verificadas y públicas
- Leer estadísticas y settings del sitio
- Crear una donación (intent, queda no verificada)
- Crear una aplicación de voluntario (queda pending)

No puede:

- Leer datos sin publicar
- Modificar nada existente
- Crear necesidades, eventos, noticias
- Leer otras aplicaciones de voluntario o donaciones no verificadas

7.4 Strategy de archivos en Supabase Storage

Buckets:

Bucket	Public	Contenido
equipment	Sí	Fotos de necesidades de equipo
events	Sí	Fotos de eventos
posts	Sí	Fotos de noticias e incidentes
gallery	Sí	Galería general de la central
volunteers	No	Documentos privados de voluntarios (post-MVP)
donations	No	Comprobantes de donaciones (privado, solo admin)

Convención de paths:

```
equipment/{equipment_need_id}/cover.jpg
equipment/{equipment_need_id}/gallery-001.jpg
events/{event_id}/cover.jpg
posts/{post_id}/cover.jpg
posts/{post_id}/gallery-001.jpg
```

Políticas de Storage:

- equipment, events, posts, gallery: lectura pública sin restricción, escritura solo para admin (verificado en Server Action antes de subir).
- volunteers, donations: lectura y escritura solo para admin.

7.5 Migraciones aditivas

Regla: todas las migraciones son aditivas. Nunca se rompe el schema existente.

■ Permitido:

- CREATE TABLE
- ALTER TABLE ADD COLUMN (con DEFAULT si NOT NULL)
- CREATE INDEX CONCURRENTLY
- CREATE POLICY
- INSERT de datos iniciales

■ Prohibido sin coordinar:

- DROP TABLE O DROP COLUMN
- ALTER COLUMN cambiando tipo
- Renombrar columnas

Si es absolutamente necesario un cambio destructivo, se hace en 3 pasos:

1. Agregar nueva columna/tabla.
2. Migrar datos vía Server Action o script SQL.
3. Eliminar la vieja en una migración posterior, cuando todos los usos están confirmados eliminados.

Numeración: 00001_descripcion.sql, 00002_descripcion.sql, etc. Strictly sequential.

8. Sistema de diseño

8.1 Principios

1. **Fotografía real, no stock.** Bomberos reales, equipo real, incidentes reales. El stock se nota y mata la confianza.
2. **Números siempre visibles.** Costos, progreso, llamadas atendidas. La transparencia se mide en cifras.
3. **Un CTA dominante por sección.** No competir por atención. Cada bloque tiene una sola acción primaria en rojo.
4. **Bilingüe sin fricción.** Toggle ES/EN siempre visible. URL refleja idioma para SEO.
5. **Alto contraste.** Tono heroico exige negros profundos y blancos puros, no grises.

8.2 Paleta de colores

```
:root {  
  /* Primarios */  
  --color-fire-red: #B91C1C; /* Rojo bombero – marca, acción, urgencia */  
  --color-fire-red-dark: #7F1D1D; /* Hover, énfasis */  
  --color-fire-red-light: #FEE2E2; /* Fondo de badges críticos */  
  
  /* Neutros */  
  --color-deep-black: #0A0A0A; /* Hero, texto principal */  
  --color-soft-black: #292524; /* Texto secundario */  
  --color-warm-gray: #57534E; /* Texto terciario */  
  --color-bone: #F5F5F4; /* Fondo neutral */  
  --color-pure-white: #FFFFFF; /* Fondo claro */  
  
  /* Semánticos */  
  --color-critical: #B91C1C; /* Prioridad crítica */  
  --color-high: #D97706; /* Prioridad alta – ámbar */  
  --color-medium: #2563EB; /* Prioridad media – azul */  
  --color-low: #6B7280; /* Prioridad baja */  
  --color-fulfilled: #047857; /* Necesidad cumplida */  
  --color-fulfilled-light: #D1FAE5; /* Badge de cumplido */  
  
  /* Bordes */  
  --color-border: #E7E5E4;  
  --color-border-strong: #D6D3D1;  
}
```

8.3 Tipografía

Uso	Familia	Pesos	Ejemplo
Display, headlines, números grandes	Oswald	500, 600	"CADA SEGUNDO CUENTA"
Cuerpo, UI, formularios	Inter	400, 500, 600	"Los bomberos voluntarios..."
Código, números técnicos	JetBrains Mono	400	"#B91C1C"

Escala tipográfica:

display-xl	72px	/ 1.0	/ weight 600	/ -2% letter-spacing	/ uppercase (hero headline)
display-lg	56px	/ 1.05	/ weight 600	/ -1.5%	/ uppercase (section headlines)
display-md	40px	/ 1.1	/ weight 600	/ -1%	/ uppercase (page titles)
display-sm	28px	/ 1.15	/ weight 600	/ 0	/ uppercase (card titles)
body-xl	20px	/ 1.6	/ weight 400		(lead paragraph)
body-lg	18px	/ 1.6	/ weight 400		(sub-lead)
body-md	16px	/ 1.6	/ weight 400		(default body)
body-sm	14px	/ 1.5	/ weight 400		(small body)
body-xs	12px	/ 1.4	/ weight 400		(captions, labels)
label-sm	12px	/ 1	/ weight 500	/ 5% letter-spacing	/ uppercase (badges)

8.4 Espaciado y layout

Sistema base de 4px:

0	= 0
1	= 4px
2	= 8px
3	= 12px
4	= 16px
6	= 24px
8	= 32px
12	= 48px
16	= 64px
24	= 96px
32	= 128px

Contenedores:

```
max-w-screen-xl = 1280px (default container)
max-w-prose = 65ch (long-form content)
max-w-screen-sm = 640px (forms, narrow content)
```

Padding lateral por breakpoint:

```
mobile px-4 (16px)
tablet px-6 (24px)
desktop px-8 (32px)
```

8.5 Componentes UI clave

Botón primario (Donar, CTA):

- Fondo #B91C1C, texto blanco
- Padding 12px 24px, radius 8px, peso 500
- Hover: fondo #7F1D1D
- Focus: ring #B91C1C opacidad 40%

Botón secundario:

- Fondo transparente, borde 0.5px solid currentColor, opacidad 25%
- Texto color heredado, peso 500
- Hover: opacidad de borde a 100%

Card de necesidad:

- Fondo blanco, borde 0.5px solid #E7E5E4, radius 8px
- Cover image arriba (ratio 16:9, fallback oscuro con icono)
- Badge de prioridad
- Título display-sm
- Progreso (texto + barra)
- Costo en Oswald 600

Badge de prioridad:

- Crítica: fondo #FEE2E2, texto #991B1B
- Alta: fondo #FEF3C7, texto #92400E
- Media: fondo #DBEAFE, texto #1E40AF
- Baja: fondo #F3F4F6, texto #374151
- Cubierto: fondo #D1FAE5, texto #065F46

■ ■ Progreso barra ■
■ ■ 2 de 4 cubiertos ■
■ ■ ■
■ ■ [Donar este eq.] ■
■ ■ [Donar monetario]■

■ Descripción larga ■
■ Especificaciones técnicas (tabla) ■
■ Donantes que contribuyeron (lista) ■

8.7 Tailwind config

```
// tailwind.config.ts
import type { Config } from 'tailwindcss'

const config: Config = {
  content: ['./src/**/*.{ts,tsx}'],
  theme: {
    extend: {
      colors: {
        fire: {
          DEFAULT: '#B91C1C',
          dark: '#7F1D1D',
          light: '#FEE2E2',
        },
        bone: '#F5F5F4',
        deep: '#0A0A0A',
      },
      fontFamily: {
        sans: ['Inter', 'system-ui', 'sans-serif'],
        display: ['Oswald', 'Arial Narrow', 'sans-serif'],
        mono: ['JetBrains Mono', 'monospace'],
      },
      fontSize: {
```

```

'display-xl': ['72px', { lineHeight: '1', letterSpacing: '-0.02em' }],
'display-lg': ['56px', { lineHeight: '1.05', letterSpacing: '-0.015em' }],
'display-md': ['40px', { lineHeight: '1.1', letterSpacing: '-0.01em' }],
'display-sm': ['28px', { lineHeight: '1.15' }],
},
animation: {
  'fade-in': 'fade-in 0.3s ease-out',
},
keyframes: {
  'fade-in': {
    '0%': { opacity: '0', transform: 'translateY(8px)' },
    '100%': { opacity: '1', transform: 'translateY(0)' },
  },
},
},
},
},
plugins: [require('tailwindcss-animate')],
}

export default config

```

8.8 Accesibilidad obligatoria

Aspecto	Requisito
Contraste texto/fondo	Mínimo 4.5:1 (WCAG AA) para body, 3:1 para large text
Foco visible	Ring de color en todos los elementos interactivos
Tamaño mínimo de touch target	44x44px en móvil
Imágenes con alt	Toda imagen no decorativa tiene alt descriptivo
Forms con labels	Cada input tiene <label htmlFor> asociado
Skip nav link	En el header para saltar al main
Heading hierarchy	h1 único, h2-h6 sin saltos
Idioma del documento	<html lang={locale}> correctamente seteado

9. Convenciones de código

9.1 TypeScript

Configuración estricta obligatoria. tsconfig.json:

```
{
  "compilerOptions": {
    "target": "ES2022",
    "lib": ["dom", "dom.iterable", "esnext"],
    "allowJs": false,
    "skipLibCheck": true,
    "strict": true,
    "noEmit": true,
    "esModuleInterop": true,
    "module": "esnext",
    "moduleResolution": "bundler",
    "resolveJsonModule": true,
    "isolatedModules": true,
    "jsx": "preserve",
    "incremental": true,
    "plugins": [{ "name": "next" }],
    "paths": {
      "@/*": [".src/*"]
    },
    "noImplicitAny": true,
    "strictNullChecks": true,
    "noUncheckedIndexedAccess": true,
    "noUnusedLocals": true,
    "noUnusedParameters": true
  },
  "include": ["next-env.d.ts", "**/*.ts", "**/*.tsx", ".next/types/**/*.ts"],
  "exclude": ["node_modules"]
}
```

Reglas no negociables:

- Nunca usar `any`. Si es necesario, usar `unknown` y `narrow type`.
- Nunca usar `// @ts-ignore`. Usar `// @ts-expect-error` con comentario explicativo si es absolutamente necesario.
- Tipar siempre `returns` de funciones públicas/exportadas.
- Preferir `type` sobre `interface` excepto para extender.
- Imports absolutos con `@/`, nunca relativos profundos `../.../..`.

9.2 React y Next.js

Server Components por defecto. Solo usar `'use client'` cuando se necesite:

- Estado local (`useState`, `useReducer`)
- Efectos (`useEffect`)
- Event handlers (`onClick`, `onChange`)
- Browser APIs (`localStorage`, `window`, `document`)
- Hooks de third party que requieren cliente

Server Actions para mutaciones. Forma estándar:

```
'use server'

import { z } from 'zod'
import { revalidatePath } from 'next/cache'
import { createClient } from '@lib/supabase/server'
import { ActionResult } from '@types'

const schema = z.object({
  title_es: z.string().min(3).max(200),
  title_en: z.string().min(3).max(200),
  // ...
})

export async function createEquipmentNeed(
  formData: FormData
): Promise<ActionResult<{ id: string }>> {
  // 1. Parsear
  const raw = Object.fromEntries(formData)
  const parsed = schema.safeParse(raw)
  if (!parsed.success) {
    return { ok: false, error: 'Datos inválidos', issues: parsed.error.flatten() }
```



```
}

// 2. Verificar permisos
const supabase = await createClient()
const { data: { user } } = await supabase.auth.getUser()
if (!user) return { ok: false, error: 'No autenticado' }

const { data: admin } = await supabase
  .from('admin_users')
  .select('user_id')
  .eq('user_id', user.id)
  .single()
if (!admin) return { ok: false, error: 'No autorizado' }

// 3. Mutar
const { data, error } = await supabase
  .from('equipment_needs')
  .insert(parsed.data)
  .select('id')
  .single()

if (error) return { ok: false, error: error.message }

// 4. Revalidar
revalidatePath('/[locale]/necesidades', 'page')
revalidatePath('/admin/necesidades')

// 5. Return
return { ok: true, data: { id: data.id } }
}
```

Tipo ActionResult *estándar*:

```
// src/types/index.ts
export type ActionResult<T = void> =
  | { ok: true; data: T }
  | { ok: false; error: string; issues?: unknown }
```

9.3 Validación con Zod

Cada feature tiene su schema en `src/lib/validations/<feature>.ts`. Los schemas se reutilizan en:

- Server Actions (validación de input)
- react-hook-form (validación de formularios con `zodResolver`)
- Tipos derivados con `z.infer<typeof schema>`

Ejemplo:

```
// src/lib/validations/equipment-need.ts

import { z } from 'zod'

export const equipmentNeedSchema = z.object({
  slug: z.string().regex(/^[a-z0-9-]+$/, 'Solo minúsculas, números y guiones'),
  title_es: z.string().min(3, 'Mínimo 3 caracteres').max(200),
  title_en: z.string().min(3).max(200),
  description_es: z.string().max(2000).optional(),
  description_en: z.string().max(2000).optional(),
  category: z.enum(['epp', 'tools', 'vehicles', 'medical', 'communications', 'station', 'other']),
  priority: z.enum(['critical', 'high', 'medium', 'low']),
  estimated_cost_usd: z.coerce.number().positive().optional(),
  estimated_cost_mxn: z.coerce.number().positive().optional(),
  quantity_needed: z.coerce.number().int().positive(),
  quantity_received: z.coerce.number().int().min(0),
  published: z.coerce.boolean(),
  featured: z.coerce.boolean(),
})

export type EquipmentNeedInput = z.infer<typeof equipmentNeedSchema>
```

9.4 Estructura de un componente

```
// src/components/public/NeedCard.tsx

import Link from 'next/link'
import Image from 'next/image'
import { useLocale, useTranslations } from 'next-intl'
import { Badge } from '@components/ui/badge'
```

```
import { ProgressBar } from '@components/public/ProgressBar'

import { formatCurrency } from '@lib/utils'

import type { EquipmentNeed } from '@types'

interface NeedCardProps {
  need: EquipmentNeed
}

export function NeedCard({ need }: NeedCardProps) {
  const locale = useLocale()
  const t = useTranslations('needs')

  const title = locale === 'es' ? need.title_es : need.title_en
  const progress = need.quantity_received / need.quantity_needed

  return (
    <Link
      href={`/${locale}/necesidades/${need.slug}`}
      className="block overflow-hidden rounded-lg border border-stone-200 transition-shadow
        hover:shadow-md"
    >
      <div className="relative aspect-video bg-stone-900">
        {need.cover_image_url ? (
          <Image
            src={need.cover_image_url}
            alt={title}
            fill
            className="object-cover"
            sizes="(max-width: 768px) 100vw, 33vw"
          />
        ) : (
          <div className="flex h-full items-center justify-center text-stone-600">
            { /* Placeholder icon */ }
          </div>
        )}
      </div>
    </Link>
  )
}
```

```
<div className="p-4">

  <Badge variant={need.priority}>{t(`priority.${need.priority}`)}</Badge>

  <h3 className="mt-2 font-display text-lg font-medium">{title}</h3>

  <p className="mt-1 text-sm text-stone-500">
    {t('progress', { received: need.quantity_received, needed: need.quantity_needed })}
  </p>

  <ProgressBar value={progress} className="mt-2" />

  <p className="mt-3 font-display text-base font-semibold">
    {formatCurrency(need.estimated_cost_usd, 'USD')}
  </p>
</div>

</Link>

)

}
```

9.5 ESLint + Prettier

.eslintrc.json:

```
{
  "extends": ["next/core-web-vitals", "next/typescript"],
  "rules": {
    "@typescript-eslint/no-unused-vars": ["error", { "argsIgnorePattern": "^_" }],
    "@typescript-eslint/no-explicit-any": "error",
    "react/no-unescaped-entities": "off",
    "react-hooks/exhaustive-deps": "error"
  }
}
```

.prettierrc:

```
{
  "semi": false,
  "singleQuote": true,
  "trailingComma": "es5",
  "printWidth": 100,
```

```
"tabWidth": 2,  
"plugins": ["prettier-plugin-tailwindcss"]  
}
```

9.6 Comentarios y documentación

- **Código autodocumentado:** preferir nombres expresivos sobre comentarios.
- **JSDoc para funciones públicas complejas:** especialmente Server Actions exportadas.
- **Comentarios explican el "por qué", no el "qué".**
- **TODO con autor y fecha:** `// TODO(claude, 2026-05-26): refactor when X is fixed.`

9.7 Manejo de errores

En Server Actions: retornar `{ ok: false, error }`, nunca lanzar excepciones al cliente.

En Server Components: envolver fetches críticos en `try/catch` y mostrar `error.tsx` si falla.

En Client Components: mostrar toast de error con mensaje user-friendly.

Errores no recuperables: capturar con Sentry y mostrar `error.tsx` con opción de reload.

Nunca mostrar al usuario:

- Stack traces
- Mensajes de Supabase crudos
- IDs internos o slugs técnicos
- URLs de Storage

Siempre traducir mensajes de error a i18n keys.

10. Workflow de Git y GitHub

10.1 Filosofía

Trabajamos con dos agentes de código en paralelo durante una semana intensiva. Cada decisión optimiza para **minimizar conflictos y permitir trabajo concurrente sin pisarse**.

Regla de oro: nadie hace push directo a main. Nunca. main solo recibe commits vía Pull Request mergeado tras CI verde.

10.2 Estructura de branches

```
main → producción (auto-deploy a Vercel)

■■■■ develop → integración (auto-deploy a preview)

■ ■■■■ feat/needs ← Claude Code
■ ■■■■ feat/donations ← Claude Code
■ ■■■■ feat/events ← Codex
■ ■■■■ feat/volunteers ← Codex
■ ■■■■ fix/lang-toggle ← cualquiera
■ ■■■■ chore/deps-update ← cualquiera
```

Reglas:

- main está protegida. Solo recibe merges desde develop vía PR aprobado.
- develop es la rama de integración. Las features apuntan aquí.
- Branches efímeras: una por feature, se borran al mergear.
- Si un hotfix urgente entra a main, se hace cherry-pick a develop inmediatamente.

10.3 Convención de nombres de branches

Formato: <tipo>/<descripcion-kebab-case>

Tipos permitidos:

Tipo	Uso
feat/	Nueva funcionalidad
fix/	Corrección de bug
chore/	Mantenimiento, deps, configuración
refactor/	Cambio sin afectar comportamiento
docs/	Solo documentación
style/	Formato, no afecta lógica

Ejemplos válidos:

```
feat/equipment-needs-crud
feat/stripe-webhook
fix/lang-toggle-on-mobile
chore/upgrade-nextjs-15-1
docs/admin-guide
```

Ejemplos prohibidos:

```
mi-rama ← sin tipo
feature/Needs ← PascalCase, espacios
feat_needs ← snake_case
hotfix ← sin descripción
```

10.4 Conventional Commits

Formato obligatorio:

```
<tipo>(<scope>): <descripción en imperativo>

<cuerpo opcional explicando el porqué>

<footer opcional: BREAKING CHANGE, refs a issues>
```

Tipos: feat, fix, chore, refactor, docs, style, test, perf.

Scopes (carpeta o feature):

- needs, donations, events, volunteers, posts
- admin, auth, i18n, db, ui, seo, config

Ejemplos:

```
feat(needs): add equipment_needs CRUD with image upload
fix(i18n): handle missing translation fallback to es
chore(db): add migration for donation verification
refactor(ui): extract NeedCard into shared component
docs(readme): add deployment instructions
```

Reglas:

- Descripción en inglés (consistente con código).
- Imperativo: "add" no "added" ni "adds".
- Sin punto al final.
- Primera línea ≤72 caracteres.
- Cuerpo explica el **por qué**, no el qué.

10.5 Workflow del desarrollador (humano o agente)

Inicio de feature:

```
git checkout develop
git pull origin develop
git checkout -b feat/<nombre-feature>
pnpm install # asegurar deps actualizadas
```

Durante el trabajo:

```
# Commits pequeños y atómicos
git add src/components/public/NeedCard.tsx
git commit -m "feat(needs): add NeedCard component with progress bar"

git add src/actions/equipment-needs.ts src/lib/validations/equipment-need.ts
git commit -m "feat(needs): add createEquipmentNeed server action with validation"

# Si develop avanzó mientras trabajabas, rebase
git fetch origin
git rebase origin/develop
# Resolver conflictos si los hay
git rebase --continue
```

Al terminar:

```
# Verificar localmente
pnpm lint
pnpm typecheck
pnpm build

# Si todo verde
git push -u origin feat/<nombre-feature>

# Abrir PR contra develop desde GitHub UI
# Llenar template, esperar CI verde, mergear
```

10.6 Template de Pull Request

Guardar en `.github/pull_request_template.md`:

```
## Qué hace este PR

<!-- Una frase clara que resuma el cambio -->
```



```

## Cambios principales
-
-

## Tipo de cambio
- [ ] feat – nueva funcionalidad
- [ ] fix – corrección de bug
- [ ] chore – mantenimiento
- [ ] refactor – cambio sin afectar comportamiento
- [ ] docs – solo documentación

## Checklist obligatorio
- [ ] CI verde (lint + typecheck + build)
- [ ] Probé el flow completo en preview deploy de Vercel
- [ ] No toqué archivos de la "zona compartida" sin coordinar
- [ ] Las migraciones SQL son aditivas
- [ ] Agregué i18n keys en `es.json` Y `en.json` (si aplica)
- [ ] Las nuevas tablas tienen RLS habilitada con políticas
- [ ] Los Server Actions validan permisos antes de mutar
- [ ] No hay strings hardcodeados en UI (todo vía i18n)
- [ ] No hay console.log de debug olvidados

## Screenshots o video
<!-- Si hay cambios visuales -->

## URL de preview
<!-- Vercel deja comentario automático -->

## Notas para el reviewer
<!-- Decisiones tomadas, trade-offs, contexto adicional -->

```

10.7 Estrategia anti-conflictos para 2 agentes paralelos

Zonas calientes (alto riesgo de conflicto — coordinar antes de tocar):

Archivo / carpeta	Estrategia
src/middleware.ts	Solo uno asignado
next.config.mjs	Solo uno asignado

Archivo / carpeta	Estrategia
tailwind.config.ts	Solo uno asignado
src/lib/i18n/*.json	Cualquiera, pero solo agrega keys, nunca renombra
supabase/migrations/*	Numeración secuencial, avisar antes de crear
src/components/ui/*	Solo si shadcn no lo tiene
package.json	Cualquiera, pero avisar (conflictos en lock)
src/lib/supabase/*	Setup inicial, no agregar funciones

Zonas frías (paralelización segura):

Carpeta	Notas
src/app/[locale]/<feature>/*	Cada feature, su carpeta
src/actions/<feature>.ts	Un archivo por feature
src/components/public/<Feature>*	Componentes específicos
src/components/admin/<Feature>*	Componentes específicos
src/lib/validations/<feature>.ts	Schemas por feature

Regla del "primero en llegar":

Si dos agentes necesitan modificar el mismo archivo de zona caliente:

1. El primero abre PR, lo mergea.
2. El segundo hace `git pull origin develop`, rebase su rama, resuelve conflictos.

No paralelizar cambios en archivos compartidos. Secuencial es más rápido.

10.8 Protección de branches en GitHub

Settings → *Branches* → *Add rule para main*:

- ☒ Require a pull request before merging
- ☒ Require approvals (1 mínimo si hay otro reviewer, 0 si trabajas solo)
- ☒ Dismiss stale pull request approvals when new commits are pushed
- ☒ Require status checks to pass before merging
 - ☒ Require branches to be up to date before merging
 - ☒ Status checks: quality / Lint, typecheck, build
- ☒ Require conversation resolution before merging
- ☒ Restrict who can push to matching branches: nadie (excepto admins en caso de emergencia)

- ■ Allow force pushes: no
- ■ Allow deletions: no

Settings → *Branches* → *Add rule para develop*:

Mismas reglas que main, pero permite que cualquier colaborador mergee tras CI verde.

11. Configuración de Vercel y CI/CD

11.1 Setup inicial de Vercel (paso a paso)

1. Crear proyecto en Vercel:

1. Ir a `vercel.com` → Add New → Project
2. Importar repo de GitHub (autoriza acceso si es primera vez)
3. Selecciona "bomberos-todos-santos"
4. Framework Preset: Next.js (auto-detectado)
5. Root directory: `./` (raíz)
6. Build command: `pnpm build` (auto-detectado)
7. Output directory: `.next` (auto)
8. Install command: `pnpm install` (auto)

2. Configurar variables de entorno (*Settings* → *Environment Variables*):

```
NEXT_PUBLIC_SUPABASE_URL → URL del proyecto Supabase
NEXT_PUBLIC_SUPABASE_ANON_KEY → anon key pública
SUPABASE_SERVICE_ROLE_KEY → service role (marcar como SECRET)
NEXT_PUBLIC_SITE_URL → https://bomberos-todos-santos.org (o equivalente)

STRIPE_SECRET_KEY → sk_test_... (MVP) o sk_live_... (prod)
STRIPE_WEBHOOK_SECRET → whsec_...
NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY → pk_...

PAYPAL_CLIENT_ID → live o sandbox
PAYPAL_CLIENT_SECRET → live o sandbox
NEXT_PUBLIC_PAYPAL_CLIENT_ID → mismo client_id

RESEND_API_KEY → re_...
```

RESEND_FROM_EMAIL → no-reply@bomberos-todos-santos.org

SENTRY_DSN → DSN del proyecto Sentry

SENTRY_AUTH_TOKEN → para source maps en build

NEXT_PUBLIC_PLAUSIBLE_DOMAIN → bomberos-todos-santos.org (si usas Plausible)

Distribución por environment:

Variable	Production	Preview	Development
SUPABASE	Production project	Production project (MVP)	.env.local
STRIPE_*	Live keys	Test keys	Test keys
RESEND_*	Real domain	Real domain	onboarding@resend.dev
NEXT_PUBLIC_SITE_URL	dominio final	\$VERCEL_URL automático	localhost:3000

Nota: para MVP con sprint de una semana, usar el mismo proyecto Supabase en preview y prod evita complejidad. Diferenciar con prefijos en seed si es necesario.

3. Settings → Git:

Production Branch: main

Auto Deploy: ON (auto-deploy a producción en cada push a main)

Preview Deployments: ON para todas las branches

4. Settings → Domains:

1. Add Domain → bomberos-todos-santos.org (o el final)
2. Vercel da DNS records para configurar en el registrar
3. Subdominio para staging si quieres: staging.bomberos-todos-santos.org → develop

5. Settings → Functions:

Region: iad1 (Washington, cerca de Supabase US-east) o sfo1 si Supabase está en West.

Verificar región del proyecto Supabase y elegir la más cercana.

11.2 GitHub Actions: CI workflow

Archivo .github/workflows/ci.yml:

```
name: CI
```

```
on:
  pull_request:
  branches: [main, develop]
  push:
  branches: [main, develop]

jobs:
  quality:
    name: Lint, typecheck, build
    runs-on: ubuntu-latest
    timeout-minutes: 10

    steps:
      - name: Checkout
        uses: actions/checkout@v4

      - name: Setup pnpm
        uses: pnpm/action-setup@v3
        with:
          version: 9

      - name: Setup Node.js
        uses: actions/setup-node@v4
        with:
          node-version: 20
          cache: 'pnpm'

      - name: Install dependencies
        run: pnpm install --frozen-lockfile

      - name: Lint
        run: pnpm lint

      - name: Type check
        run: pnpm typecheck

      - name: Build

    env:
      NEXT_PUBLIC_SUPABASE_URL: ${ secrets.NEXT_PUBLIC_SUPABASE_URL }
```

```
NEXT_PUBLIC_SUPABASE_ANON_KEY: ${ secrets.NEXT_PUBLIC_SUPABASE_ANON_KEY }}  
  
run: pnpm build
```

Secrets que GitHub necesita (Settings → Secrets and variables → Actions):

```
NEXT_PUBLIC_SUPABASE_URL  
  
NEXT_PUBLIC_SUPABASE_ANON_KEY
```

Solo las dos que se necesitan en build. Vercel maneja el resto en deploy.

11.3 Flujo automatizado completo

```
1. Dev abre PR contra develop  
↓  
2. GitHub dispara CI (ci.yml)  
- pnpm install  
- pnpm lint  
- pnpm typecheck  
- pnpm build  
↓  
3. Vercel detecta el PR y crea preview deploy  
- URL única: bomberos-pr-23-bomberos.vercel.app  
- Bot comenta el PR con el link  
↓  
4. Reviewer (humano) prueba el preview, aprueba  
↓  
5. Merge a develop  
- Vercel re-despliega develop a develop.bomberos.vercel.app  
↓  
6. Al final del sprint: PR develop → main  
↓  
7. Merge a main  
- Vercel despliega a producción  
- Dominio principal apunta aquí
```

11.4 Webhooks de Stripe en preview deploys

Problema: cada preview tiene URL distinta, los webhooks de Stripe no pueden apuntar a todas.

Soluciones:

1. **Solo testear webhooks en producción** (recomendado para MVP).

2. **Stripe CLI para desarrollo local:**

```
stripe listen --forward-to localhost:3000/api/webhooks/stripe
```

3. **Endpoint de prueba en preview con secret diferente:** se configura uno en Stripe Dashboard apuntando a *.vercel.app con un wildcard manual.

Para el MVP, **opción 1 + opción 2** es suficiente.

11.5 Rollback en caso de problemas

Si producción falla:

1. Vercel Dashboard → Deployments
2. Buscar el deploy estable anterior
3. Click ... → Promote to Production
4. Vercel apunta el dominio a ese deploy en <30 segundos

Si la causa es código, también:

```
git checkout main
git revert <hash-del-commit-malo>
git push origin main
# Vercel redespiega automáticamente
```

Nunca hacer `git reset --hard` **en main**. Siempre revert para mantener historia limpia.

12. Plan de sprint (7 días)

12.1 Día 0 — Domingo: setup compartido

Quién: una persona (tú o un solo agente). No paralelizar aún.

Pre-requisitos antes de empezar:

- Cuenta en GitHub
- Cuenta en Vercel conectada a GitHub

- Cuenta en Supabase
- Cuenta en Resend
- Logo de los bomberos (SVG si es posible)
- 5-10 fotos reales (preferiblemente)

Tareas:

1. Crear repo bomberos-todos-santos en GitHub.
2. Configurar .gitignore para Node.js + Next.js.
3. Crear proyecto Supabase, copiar URL y keys.
4. Correr schema.sql en SQL Editor de Supabase.
5. Crear buckets de Storage: equipment, events, posts, gallery (public); volunteers, donations (private).
6. Configurar políticas básicas de Storage.
7. `npx create-next-app@latest bomberos-todos-santos --typescript --tailwind --app --src-dir --import-alias "@/*"`.
8. Instalar deps adicionales (ver lista en sección 5.3).
9. Configurar shadcn/ui: `npx shadcn-ui@latest init`.
10. Configurar next-intl con locales ES (default) y EN.
11. Crear estructura de carpetas según sección 6.2.
12. Generar tipos de Supabase: `pnpm db:types`.
13. Crear `lib/supabase/client.ts`, `server.ts`, `admin.ts`.
14. Crear `middleware.ts` con `i18n + auth guard` para `/admin/*`.
15. Crear layout público (Header + Footer básico) y layout admin (Sidebar).
16. Crear página de login admin con magic link.
17. Insertar usuario admin inicial en `auth.users + admin_users` (desde Supabase Studio).
18. Conectar Vercel al repo, configurar variables de entorno.
19. Crear branch `develop`, proteger ambas branches.
20. Push inicial a `main`, verificar deploy verde.

Entregable: repo deployado a Vercel mostrando "Hello World" bilingüe, con admin login funcional y CI verde.

Tiempo estimado: 6-8 horas concentradas.

12.2 Día 1 — Lunes: features paralelas iniciales

Claude Code → `feat/equipment-needs` (**parte 1**):

- Server Actions: `createEquipmentNeed`, `updateEquipmentNeed`, `deleteEquipmentNeed`, `toggleFeatured`, `togglePublished`.
- Validation schema `equipment-need.ts`.

- Componente `ImageUploader.tsx` (sube a Supabase Storage).
- Componente `BilingualTextField.tsx`.
- Página `/admin/necesidades` con `DataTable`.
- Página `/admin/necesidades/nueva` con `NeedForm`.
- Página `/admin/necesidades/[id]` con `NeedForm` pre-cargado.

Codex → `feat/events`:

- Server Actions de events: `createEvent`, `updateEvent`, `deleteEvent`.
- Validation schema `event.ts`.
- Componente `EventForm.tsx`.
- Página `/admin/eventos` con `DataTable`.
- Páginas `/admin/eventos/nuevo` y `/admin/eventos/[id]`.
- Página pública `/[locale]/eventos` con lista de events publicados.
- Página `/[locale]/eventos/[slug]` con detalle.

Coordinación necesaria: ambos consumen `ImageUploader` (Claude Code lo crea, Codex lo usa después).

12.3 Día 2 — Martes

Claude Code (continúa `feat/equipment-needs`):

- Componente `NeedCard.tsx`.
- Componente `NeedDetail.tsx`.
- Componente `ProgressBar.tsx`.
- Componente `NeedsFilter.tsx` (filtros por categoría y prioridad).
- Página pública `/[locale]/necesidades` con grid filtrable.
- Página `/[locale]/necesidades/[slug]` con detalle, galería, CTA.
- Merge PR a `develop`, despliegue a `preview`.

Codex → `feat/volunteers`:

- Server Actions: `submitVolunteerApplication`, `approveVolunteer`, `rejectVolunteer`.
- Validation schema `volunteer.ts`.
- Componente `VolunteerForm.tsx` (formulario público).
- Componente `VolunteerReviewCard.tsx` (admin).
- Email templates: `volunteer-received.tsx`, `volunteer-decision.tsx`.
- Página `/[locale]/voluntarios` con info + form.
- Página `/admin/voluntarios` con lista filtrable.
- Integración Resend para emails automáticos.

12.4 Día 3 — Miércoles

Claude Code → feat/donations:

- Server Actions: createDonationIntent, verifyDonation, toggleDonationPublic.
- Validation schema donation.ts.
- Componente DonationIntentForm.tsx.
- Componente DonationVerifyCard.tsx.
- Página /[locale]/donar con métodos: transferencia (datos bancarios visibles), PayPal button, Stripe button (preparado, off).
- Página /[locale]/donar/gracias post-submit.
- Página /admin/donaciones con filtros y verificación.
- Email automático a admin cuando alguien crea donation intent.

Codex → feat/posts:

- Server Actions de posts.
- Validation schema post.ts.
- Componente PostCard.tsx.
- Componente PostForm.tsx (con rich text simple para contenido).
- Página /[locale]/noticias con lista paginada.
- Página /[locale]/noticias/[slug] con detalle, galería.
- Páginas admin: /admin/noticias, /admin/noticias/nueva, /admin/noticias/[id].

12.5 Día 4 — Jueves

Claude Code (continúa feat/donations + nueva feat/transparency):

- Componente DonorWall.tsx.
- Página /[locale]/transparencia con muro filtrable, agradecimientos, métricas.
- Server Action getPublicDonations con paginación.
- Webhooks Route Handlers:
 - /api/webhooks/stripe/route.ts (preparado, verificación de signature).
 - /api/webhooks/paypal/route.ts (preparado).
- Integración Stripe SDK (stub si stripe_enabled = false en settings).

Codex → feat/home-and-about:

- Página home /[locale]/page.tsx:
 - HeroSection con CTA dual.
 - StatsBar con números del año actual.
 - Sección "Necesidades urgentes" (3 destacadas).

- Sección "Últimas noticias" (3 más recientes).
- Sección "Próximos eventos" (3 próximos).
- Footer rico.
- Página / [locale] / nosotros con historia, equipo, mapa.
- Páginas admin: /admin/stats (editor de station_stats por año), /admin/settings (editor de site_settings).

12.6 Día 5 — Viernes: integración y polish

Ambos agentes mergean sus PRs pendientes a develop.

Trabajo conjunto del día:

- QA bilingüe completo en preview deploy (revisar cada página en ES y EN).
- Responsive check exhaustivo: mobile (375px), tablet (768px), desktop (1280px+).
- SEO setup:
 - Metadata para cada página (title, description, og:image).
 - sitemap.ts que genere sitemap dinámico.
 - robots.ts.
 - Open Graph images dinámicas con /api/og/[type].
 - Schema.org JSON-LD para organización y eventos.
- Lighthouse audit en preview, fix de issues críticos:
 - Performance ≥ 90
 - Accessibility ≥ 95
 - Best Practices ≥ 95
 - SEO ≥ 95
- Setup Sentry para error tracking.
- Setup Plausible o Vercel Analytics.
- Cargar contenido real inicial:
 - 5 necesidades de equipo con fotos
 - 1-2 eventos próximos
 - 1-2 noticias / agradecimientos
 - Datos de station_stats del año actual
 - Site settings: contacto, redes, datos bancarios

12.7 Día 6 — Sábado: go-live

Tareas finales:

1. PR final develop → main con changelog completo.

2. Merge → deploy automático a producción.

3. Configurar dominio en Vercel.

4. Validar DNS, SSL.

5. Smoke test completo en producción:
 - Crear necesidad de prueba, verla pública.
 - Crear donation intent, verificarla.
 - Aplicar como voluntario, aprobar.
 - Verificar emails llegando.

6. Grabar 3 Loom videos de capacitación admin:
 - "Cómo subir una nueva necesidad de equipo" (8 min)
 - "Cómo verificar y publicar una donación" (5 min)
 - "Cómo aprobar voluntarios y crear eventos" (10 min)

7. Sesión en vivo con el bombero admin (1 hora) para hacer ejercicios prácticos.

8. Documentar URLs, credenciales (en gestor de contraseñas, no en código).

9. Difusión inicial: redes sociales, WhatsApp comunitario, newsletter inicial.

12.8 Resumen del sprint en tabla

Día	Claude Code	Codex	Compartido
0	—	—	Setup completo
1	feat/needs (admin CRUD)	feat/events	—
2	feat/needs (público)	feat/volunteers	—
3	feat/donations	feat/posts	—
4	feat/transparency + webhooks	feat/home-and-about	—
5	—	—	QA, SEO, polish, contenido
6	—	—	Go-live, capacitación, difusión

13. Briefings para agentes

13.1 Briefing maestro (compartido para Claude Code y Codex)

Copiar y pegar al inicio de cada sesión nueva con cualquier agente:

Contexto del proyecto

Sitio web para los Bomberos Voluntarios de Todos Santos, BCS. Permite:

- Mostrar necesidades de equipo específicas con costo, prioridad y progreso
- Recibir donaciones (transferencia, PayPal; Stripe preparado pero off)
- Registro de voluntarios con aprobación admin
- Eventos, cursos, noticias, transparencia pública

Audiencia bilingüe (español default, inglés disponible).

Stack

- Next.js 15 (App Router) + TypeScript estricto
- Tailwind + shadcn/ui
- Supabase: Postgres + Auth + Storage + RLS
- next-intl para i18n
- Zod + react-hook-form para forms
- Resend para email transaccional
- Deploy: Vercel
- Package manager: pnpm

Convenciones obligatorias

Git

- NUNCA push directo a main. NUNCA.
- Trabajar siempre en branch feat/<nombre-kebab> partiendo de develop
- Commits: Conventional Commits → feat(scope): descripción en imperativo
- PR contra develop, no contra main
- Esperar CI verde antes de mergear

Código

- Server Actions en src/actions/<feature>.ts, todos retornan { ok, data?, error? }
- Server Actions validan permisos con is_admin() antes de mutar
- Validación con Zod siempre, schemas en src/lib/validations/<feature>.ts
- Componentes: PascalCase, ubicados según público/admin/forms
- Carpetas: kebab-case
- DB: snake_case_plural para tablas, snake_case para columnas
- Tipos TS: PascalCase, derivados de src/types/database.ts

Internacionalización

- TODO texto visible en src/lib/i18n/es.json y en.json
- Si agregas key en uno, agrégala en el otro
- Nunca hardcodear strings de UI
- Contenido dinámico bilingüe: campos title_es + title_en en DB

Seguridad

- RLS habilitada en TODAS las tablas
- Lecturas públicas solo si published = true
- Mutaciones solo desde Server Actions verificando admin
- Service role key SOLO en código server (nunca en client components)

Estilo visual

- Paleta: rojo bombero #B91C1C, negro #0A0A0A, hueso #F5F5F4
- Acentos: ámbar #D97706 (alta prioridad), verde #047857 (cubierto)
- Tipografía: Oswald para headlines, Inter para cuerpo
- Tono: heroico, fotografía real, alto contraste

Estructura del repo (NO INVENTAR carpetas)

src/

- app/[locale]/... ← páginas públicas bilingües
- app/admin/... ← admin solo en español
- app/api/webhooks/... ← solo para Stripe/PayPal
- actions/ ← Server Actions
- components/
 - ■■■ ui/ ← shadcn primitives
 - ■■■ layout/ ← header, footer, sidebars
 - ■■■ public/ ← componentes de páginas públicas
 - ■■■ admin/ ← componentes del admin
 - ■■■ forms/ ← formularios reutilizables
- lib/
 - ■■■ supabase/ ← clients server/browser/admin
 - ■■■ validations/ ← Zod schemas
 - ■■■ i18n/ ← config + es.json + en.json
 - ■■■ utils.ts

```
■■■ types/database.ts ← auto-generado, NO editar

## Zonas compartidas – NO TOCAR sin coordinar

- src/middleware.ts
- next.config.mjs
- tailwind.config.ts
- package.json (puedes agregar deps pero avisa)
- src/lib/supabase/* (solo cambios estructurales)
- supabase/migrations/* (numeración secuencial, avisar)

## Antes de empezar

git checkout develop
git pull origin develop
git checkout -b feat/<nombre>
pnpm install

## Cuando termines

pnpm lint
pnpm typecheck
pnpm build

git add .
git commit -m "feat(scope): descripción"
git push -u origin feat/<nombre>

# Abre PR contra develop
```

13.2 Briefings específicos por feature

Briefing para feat/equipment-needs

```
## Tu feature: equipment_needs CRUD + páginas públicas

**Branch:** feat/equipment-needs

**Objetivo:** sistema completo de gestión y visualización de necesidades de equipo.

**Entregables:**

1. **Server Actions** en src/actions/equipment-needs.ts:
```

- createEquipmentNeed(formData)
- updateEquipmentNeed(id, formData)
- deleteEquipmentNeed(id)
- toggleFeatured(id, featured)
- togglePublished(id, published)

Todos retornan ActionResult<T>.

2. ****Validation schema**** en src/lib/validations/equipment-need.ts:

- Zod schema con todos los campos según schema.sql
- Tipos derivados: EquipmentNeedInput, EquipmentNeedUpdate

3. ****Componentes públicos**** en src/components/public/:

- NeedCard.tsx (preview en grid)
- NeedDetail.tsx (página de detalle)
- ProgressBar.tsx (reutilizable)
- NeedsFilter.tsx (categoría + prioridad + status)

4. ****Componentes admin**** en src/components/admin/:

- NeedForm.tsx (crear/editar bilingüe)
- NeedsDataTable.tsx
- ImageUploader.tsx (sube a Supabase Storage bucket 'equipment')
- BilingualTextField.tsx

5. ****Páginas públicas****:

- src/app/[locale]/necesidades/page.tsx (grid filtrable)
- src/app/[locale]/necesidades/[slug]/page.tsx (detalle)

6. ****Páginas admin****:

- src/app/admin/necesidades/page.tsx (lista)
- src/app/admin/necesidades/nueva/page.tsx (form crear)
- src/app/admin/necesidades/[id]/page.tsx (form editar)

7. ****i18n keys**** en es.json y en.json:

- needs.title, needs.subtitle
- needs.priority.critical/high/medium/low
- needs.category.epp/tools/vehicles/medical/communications/station/other
- needs.status.open/partial/fulfilled
- needs.progress (con interpolación {received}/{needed})


```
- needs.donate_this, needs.donate_money
- needs.empty, needs.filter_all
- admin.needs.create_title, admin.needs.edit_title, etc.

**Criterios de aceptación:**
- [ ] CRUD funcional en admin
- [ ] Subida y borrado de imágenes funciona
- [ ] Páginas públicas renderizan en ES y EN
- [ ] Filtros funcionan sin reload (URL params)
- [ ] RLS protege correctamente (probado con usuario no-admin)
- [ ] Form validation con mensajes en ambos idiomas
- [ ] Responsive: mobile, tablet, desktop
- [ ] Lighthouse: páginas públicas con perf ≥90
- [ ] No console.errors en desarrollo
- [ ] Probado en preview deploy de Vercel

**Pistas importantes:**
- Generar slug automáticamente del title_es con utility slugify
- En NeedCard, usar locale para mostrar title_es o title_en
- En página detalle: SEO metadata bilingüe con generateMetadata
- ImageUploader debe permitir reordenar y borrar imágenes existentes
- Progress bar visual: <50% rojo, 50-99% ámbar, 100% verde
```

Briefing para feat/events

```
## Tu feature: events + páginas públicas

**Branch:** feat/events

**Objetivo:** gestión de eventos de recaudación, cursos y comunidad.

**Entregables:**

1. **Server Actions** en src/actions/events.ts:
- createEvent, updateEvent, deleteEvent, togglePublished

2. **Validation schema** en src/lib/validations/event.ts:
- Schema con starts_at, ends_at, type enum, etc.
- Validar que ends_at sea >= starts_at si está presente
```

```

3. Componentes en src/components/public/ y admin/:

- EventCard.tsx
- EventDetail.tsx
- EventForm.tsx
- EventsList.tsx (con filtro próximos/pasados)

4. Páginas:

- /[locale]/eventos (lista con tabs: próximos | pasados | cursos)
- /[locale]/eventos/[slug] (detalle)
- /[locale]/cursos (filtro events con type='course')
- /admin/eventos (lista)
- /admin/eventos/nuevo
- /admin/eventos/[id]

5. i18n keys:

- events.upcoming, events.past, events.courses
- events.type.fundraiser/course/community/training
- events.register (CTA si registration_url presente)
- events.starts_at, events.ends_at, events.location, events.capacity

Criterios de aceptación:

- [ ] CRUD completo
- [ ] Tabs próximos/pasados funcionan
- [ ] DatePicker bilingüe en form (date-fns con locale)
- [ ] Mapa embed si location es link de Google Maps
- [ ] Botón "Add to calendar" en detalle (genera .ics)
- [ ] Page web shareables con OG image dinámica
- [ ] RLS, responsive, bilingüe completo

```

Briefing para feat/donations

```

## Tu feature: donations + transparencia + webhooks

Branch: feat/donations

Objetivo: sistema de donaciones múltiples métodos + muro de transparencia.

Entregables:

```

1. ****Server Actions**** en src/actions/donations.ts:
 - createDonationIntent (público, queda is_verified=false)
 - verifyDonation (admin, marca is_verified=true y aplica al need si aplica)
 - togglePublic (admin, marca is_public)
 - getPublicDonations (paginado para muro)
 2. ****Validation schema****:
 - amount, currency, type, equipment_need_id opcional, donor_country
 - donor_email obligatorio si quiere recibo
 3. ****Componentes públicos****:
 - DonationIntentForm.tsx (pre-seleccionable por need)
 - DonorWall.tsx (muro de donaciones públicas)
 - PaymentMethodCard.tsx (transferencia, PayPal, Stripe)
 4. ****Componentes admin****:
 - DonationVerifyCard.tsx
 - DonationsDataTable.tsx
 5. ****Webhooks****:
 - src/app/api/webhooks/stripe/route.ts:
 - Verifica firma con stripe.webhooks.constructEvent
 - Procesa charge.succeeded → crea donation con is_verified=true
 - src/app/api/webhooks/paypal/route.ts (similar)
 6. ****Páginas****:
 - /[locale]/donar (métodos + form)
 - /[locale]/donar/gracias
 - /[locale]/transparencia (muro + métricas)
 - /admin/donaciones (lista con filtros pending/verified)
 7. ****i18n keys****:
 - donations.methods.transfer/paypal/stripe
 - donations.bank_info (revelar datos)
 - donations.thank_you, donations.public_optin
 - transparency.title, transparency.donors_count, transparency.total
- **Criterios de aceptación****

- [] Form de intent funciona y notifica admin por email
- [] Stripe webhook con verificación de firma
- [] PayPal SDK button funcional (sandbox para preview)
- [] Stripe deshabilitado por default vía site_settings.stripe_enabled
- [] Muro público solo muestra is_public=true AND is_verified=true
- [] Admin puede vincular donación a equipment_need y actualizar quantity_received
- [] Email de gracias automático al donante

Briefing para feat/volunteers

```
## Tu feature: volunteers application + admin review

**Branch:** feat/volunteers

**Objetivo:** captación y gestión de voluntarios.

**Entregables:**

1. **Server Actions**:
  - submitVolunteerApplication (público, queda status='pending')
  - approveVolunteer(id, notes)
  - rejectVolunteer(id, notes)

2. **Validation schema**:
  - full_name, email, phone, age (16-90), city, skills (array), availability, motivation
  - email format, phone format mexicano y extranjero

3. **Componentes**:
  - VolunteerForm.tsx
  - VolunteerReviewCard.tsx (admin)
  - VolunteersList.tsx (admin con filtros pending/approved/rejected)

4. **Email templates**:
  - volunteer-received.tsx (al voluntario, "recibimos tu aplicación")
  - volunteer-decision.tsx (al voluntario, aprobado o rechazado)
  - admin-notification.tsx (a admin cuando llega aplicación nueva)

5. **Páginas**:
  - /[locale]/voluntarios (info + form)
  - /admin/voluntarios (lista)
```

```

- /admin/voluntarios/[id] (detalle + acción)

6. **i18n**:
- volunteers.title, volunteers.requirements, volunteers.benefits
- volunteers.form.* (cada campo)
- volunteers.skills.* (lista de habilidades)
- emails.volunteer.* (asuntos y cuerpos)

**Criterios de aceptación:**
- [ ] Form valida y submitea correctamente
- [ ] Email automático al voluntario y admin
- [ ] Admin puede aprobar/rechazar con notas
- [ ] Estados visualmente diferenciados
- [ ] RLS: volunteers no se pueden leer públicamente

```

Briefing para feat/posts

```

## Tu feature: posts (noticias, incidentes, agradecimientos)

**Branch:** feat/posts

**Objetivo:** blog/noticias del sitio.

**Entregables:**

1. **Server Actions** estándar CRUD.

2. **Validation schema** con types: news/incident/thanks/announcement.

3. **Componentes**:
- PostCard.tsx
- PostDetail.tsx
- PostForm.tsx (con rich text simple para content)
- PostsList.tsx (paginado)

4. **Páginas**:
- /[locale]/noticias (lista con filtro por type)
- /[locale]/noticias/[slug]
- /admin/noticias + nueva + [id]

5. **i18n**:

```

```

- posts.title, posts.type.news/incident/thanks/announcement
- posts.published_at, posts.author
- posts.related_posts

**Criterios de aceptación:**

- [ ] Rich text content editor en form (puede ser textarea con markdown o un editor simple)
- [ ] Slug auto-generado del title_es
- [ ] Lista paginada
- [ ] Detalle con galería de imágenes
- [ ] OG image dinámica usando cover_image_url
- [ ] Schema.org Article JSON-LD

```

Briefing para feat/home-and-about

```

## Tu feature: home, nosotros, stats admin, settings admin

**Branch:** feat/home-and-about

**Objetivo:** página principal y secciones de soporte.

**Entregables:**

1. **Home** /[locale]/page.tsx:
  - HeroSection: badge + headline + descripción + CTAs duales + StatsBar
  - Sección "Necesidades urgentes": 3 needs con featured=true
  - Sección "Próximos eventos": 3 events más cercanos
  - Sección "Últimas noticias": 3 posts más recientes
  - Sección "Cómo ayudar": 3 cards (donar, voluntario, equipo)
  - Footer rico con datos de contacto

2. **Página nosotros**:
  - Historia (texto largo, editable desde admin/settings o post fijo)
  - Equipo (galería con caras)
  - Ubicación (mapa embed)
  - Contacto (formulario simple)

3. **Admin stats** /admin/stats/page.tsx:
  - Editor de station_stats por año
  - Form: year, calls_attended, active_volunteers, training_hours, community_events

```

- Lista de años pasados con edición inline
4. ****Admin settings**** /admin/settings/page.tsx:
- Editor de site_settings (singleton)
 - Campos: contact_email, phone, emergency_phone, address, maps_url
 - Redes sociales: facebook, instagram, whatsapp
 - Datos bancarios: bank_name, clabe, account, swift, paypal_url
 - Toggle stripe_enabled
- **Criterios de aceptación:****
- [] Home arriba del fold conmueve y convierte
 - [] Stats se cargan dinámicamente del año actual
 - [] Featured needs ordenadas por display_order
 - [] Settings se reflejan inmediatamente en footer y /donar
 - [] Mapa embed funciona con maps_url
 - [] Lighthouse home: perf ≥85, todo lo demás ≥95

14. Patrones de implementación

14.1 Cliente de Supabase: 3 variantes

Browser client (src/lib/supabase/client.ts):

```
import { createBrowserClient } from '@supabase/ssr'
import type { Database } from '@types/database'

export function createClient() {
  return createBrowserClient<Database>(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!
  )
}
```

Server client (src/lib/supabase/server.ts) — para Server Components y Server Actions:

```
import { createServerClient } from '@supabase/ssr'
import { cookies } from 'next/headers'
```

```
import type { Database } from '@types/database'

export async function createClient() {
  const cookieStore = await cookies()

  return createServerClient<Database>(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
    process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
    {
      cookies: {
        getAll() {
          return cookieStore.getAll()
        },
        setAll(cookiesToSet) {
          try {
            cookiesToSet.forEach(({ name, value, options }) =>
              cookieStore.set(name, value, options)
            )
          } catch {
            // Server Component: cookies set vía middleware
          }
        },
      },
    },
  )
}
```

Admin client (src/lib/supabase/admin.ts) — service role, solo server, bypassa RLS:

```
import { createClient as createSupabaseClient } from '@supabase/supabase-js'
import type { Database } from '@types/database'

// Solo importar en código server (server actions, API routes)
// Nunca exponer al cliente

export function createAdminClient() {
  return createSupabaseClient<Database>(
    process.env.NEXT_PUBLIC_SUPABASE_URL!,
```



```
process.env.SUPABASE_SERVICE_ROLE_KEY!,
{
  auth: { persistSession: false },
}
)
```

14.2 Middleware para i18n + auth

src/middleware.ts:

```
import { NextRequest, NextResponse } from 'next/server'
import createIntlMiddleware from 'next-intl/middleware'
import { routing } from '@lib/i18n/routing'
import { createServerClient } from '@supabase/ssr'

const intlMiddleware = createIntlMiddleware(routing)

export async function middleware(request: NextRequest) {
  const { pathname } = request.nextUrl

  // Skip i18n for admin and API
  if (pathname.startsWith('/admin') || pathname.startsWith('/api')) {
    if (pathname.startsWith('/admin') && pathname !== '/admin/login') {
      // Auth guard
      const response = NextResponse.next()
      const supabase = createServerClient(
        process.env.NEXT_PUBLIC_SUPABASE_URL!,
        process.env.NEXT_PUBLIC_SUPABASE_ANON_KEY!,
        {
          cookies: {
            getAll: () => request.cookies.getAll(),
            setAll: (cookiesToSet) => {
              cookiesToSet.forEach(({ name, value, options }) =>
                response.cookies.set(name, value, options)
              )
            },
          },
        },
      )
    }
  }
}
```

```
    },
  }
)

const { data: { user } } = await supabase.auth.getUser()
if (!user) {
  return NextResponse.redirect(new URL('/admin/login', request.url))
}

// Verify admin role
const { data: admin } = await supabase
  .from('admin_users')
  .select('user_id')
  .eq('user_id', user.id)
  .single()

if (!admin) {
  await supabase.auth.signOut()
  return NextResponse.redirect(new URL('/admin/login', request.url))
}

return response
}

return NextResponse.next()
}

// Apply i18n for public routes
return intlMiddleware(request)
}

export const config = {
  matcher: [
    '/((?!api|_next/static|_next/image|favicon.ico|.*\\.\\.\\.)*.*)',
  ],
}
```

14.3 Patrón de Server Action completo

```
// src/actions/equipment-needs.ts

'use server'

import { revalidatePath } from 'next/cache'
import { redirect } from 'next/navigation'
import { createClient } from '@lib/supabase/server'
import { createAdminClient } from '@lib/supabase/admin'
import { equipmentNeedSchema } from '@lib/validations/equipment-need'
import { slugify } from '@lib/utils'
import type { ActionResult } from '@types'

async function requireAdmin() {
  const supabase = await createClient()
  const { data: { user } } = await supabase.auth.getUser()
  if (!user) {
    return { error: 'No autenticado' as const }
  }
  const { data: admin } = await supabase
    .from('admin_users')
    .select('user_id')
    .eq('user_id', user.id)
    .single()
  if (!admin) {
    return { error: 'No autorizado' as const }
  }
  return { user }
}

export async function createEquipmentNeed(
  formData: FormData
): Promise<ActionResult<{ id: string }>> {
  const auth = await requireAdmin()
  if ('error' in auth) return { ok: false, error: auth.error }

  const raw = Object.fromEntries(formData)
  const parsed = equipmentNeedSchema.safeParse(raw)
```

```
if (!parsed.success) {
  return {
    ok: false,
    error: 'Datos inválidos',
    issues: parsed.error.flatten().fieldErrors,
  }
}

const slug = parsed.data.slug || slugify(parsed.data.title_es)
const admin = createAdminClient()
const { data, error } = await admin
  .from('equipment_needs')
  .insert({ ...parsed.data, slug })
  .select('id')
  .single()

if (error) return { ok: false, error: error.message }

revalidatePath('/[locale]/necesidades', 'page')
revalidatePath('/admin/necesidades')

return { ok: true, data: { id: data.id } }
}

export async function updateEquipmentNeed(
  id: string,
  formData: FormData
): Promise<ActionResult<void>> {
  const auth = await requireAdmin()
  if ('error' in auth) return { ok: false, error: auth.error }

  const raw = Object.fromEntries(formData)
  const parsed = equipmentNeedSchema.partial().safeParse(raw)
  if (!parsed.success) {
    return { ok: false, error: 'Datos inválidos', issues: parsed.error.flatten() }
  }

  const admin = createAdminClient()
```

```

const { error } = await admin
  .from('equipment_needs')
  .update(parsed.data)
  .eq('id', id)

if (error) return { ok: false, error: error.message }

revalidatePath('/[locale]/necesidades', 'page')
revalidatePath(`/[locale]/necesidades/${parsed.data.slug || ''}`)
revalidatePath('/admin/necesidades')

return { ok: true, data: undefined }
}

export async function deleteEquipmentNeed(id: string): Promise<ActionResult<void>> {
  const auth = await requireAdmin()
  if ('error' in auth) return { ok: false, error: auth.error }

  const admin = createAdminClient()
  const { error } = await admin.from('equipment_needs').delete().eq('id', id)
  if (error) return { ok: false, error: error.message }

  revalidatePath('/[locale]/necesidades', 'page')
  revalidatePath('/admin/necesidades')

  return { ok: true, data: undefined }
}

```

14.4 Patrón de Server Component para lectura

```

// src/app/[locale]/necesidades/page.tsx

import { createClient } from '@lib/supabase/server'
import { getTranslations, setRequestLocale } from 'next-intl/server'
import { NeedCard } from '@components/public/NeedCard'
import { NeedsFilter } from '@components/public/NeedsFilter'

interface PageProps {
  params: Promise<{ locale: string }>
  searchParams: Promise<{ category?: string; priority?: string }>
}

```

```
}

export async function generateMetadata({ params }: PageProps) {
  const { locale } = await params
  const t = await getTranslations({ locale, namespace: 'needs' })
  return {
    title: t('meta.title'),
    description: t('meta.description'),
  }
}

export default async function NeedsPage({ params, searchParams }: PageProps) {
  const { locale } = await params
  const { category, priority } = await searchParams
  setRequestLocale(locale)

  const supabase = await createClient()
  let query = supabase
    .from('equipment_needs')
    .select('*')
    .eq('published', true)
    .neq('status', 'archived')
    .order('priority', { ascending: true })
    .order('display_order', { ascending: true })

  if (category) query = query.eq('category', category)
  if (priority) query = query.eq('priority', priority)

  const { data: needs, error } = await query

  if (error) {
    throw new Error('No se pudieron cargar las necesidades')
  }

  const t = await getTranslations({ locale, namespace: 'needs' })

  return (
    <main className="container mx-auto px-4 py-12">
      <h1 className="font-display text-display-lg uppercase">{t('title')}</h1>
    </main>
  )
}
```

```

<NeedsFilter />

{needs && needs.length > 0 ? (
  <div className="mt-8 grid grid-cols-1 gap-6 md:grid-cols-2 lg:grid-cols-3">
    {needs.map((need) => (
      <NeedCard key={need.id} need={need} />
    ))}
  </div>
) : (
  <p className="mt-8 text-center text-stone-500">{t('empty')}</p>
)}
</main>
)
}

```

14.5 Patrón de formulario con react-hook-form + Zod + Server Action

```

'use client'

import { useTransition } from 'react'
import { useForm } from 'react-hook-form'
import { zodResolver } from '@hookform/resolvers/zod'
import { useTranslations } from 'next-intl'

import { equipmentNeedSchema, type EquipmentNeedInput } from
'@/lib/validations/equipment-need'
import { createEquipmentNeed } from '@actions/equipment-needs'
import { useToast } from '@components/ui/use-toast'

export function NeedForm() {
  const t = useTranslations('admin.needs')
  const { toast } = useToast()
  const [pending, startTransition] = useTransition()

  const form = useForm<EquipmentNeedInput>({
    resolver: zodResolver(equipmentNeedSchema),
    defaultValues: {
      title_es: '',
      title_en: '',

```

```
category: 'epp',
priority: 'medium',
quantity_needed: 1,
quantity_received: 0,
published: false,
featured: false,
},
})

function onSubmit(data: EquipmentNeedInput) {
const formData = new FormData()
Object.entries(data).forEach(([k, v]) => formData.append(k, String(v ?? '')))

startTransition(async () => {
const result = await createEquipmentNeed(formData)
if (result.ok) {
toast({ title: t('created') })
form.reset()
} else {
toast({ title: t('error'), description: result.error, variant: 'destructive' })
}
})
}

return (
<form onSubmit={form.handleSubmit(onSubmit)} className="space-y-6">
  /* Bilingual fields */
  <div className="grid grid-cols-2 gap-4">
    <Input {...form.register('title_es')} label={t('title_es')} />
    <Input {...form.register('title_en')} label={t('title_en')} />
  </div>

  /* ... más campos ... */

  <Button type="submit" disabled={pending}>
    {pending ? t('saving') : t('save')}
  </Button>
```



```
</form>

)

}
```

14.6 Patrón de upload de imágenes

```
'use client'

import { useState } from 'react'
import { createClient } from '@lib/supabase/client'
import { Button } from '@components/ui/button'

interface ImageUploaderProps {
  bucket: 'equipment' | 'events' | 'posts' | 'gallery'
  recordId: string
  onUpload: (url: string) => void
}

export function ImageUploader({ bucket, recordId, onUpload }: ImageUploaderProps) {
  const [uploading, setUploading] = useState(false)
  const supabase = createClient()

  async function handleUpload(e: React.ChangeEvent<HTMLInputElement>) {
    const file = e.target.files?.[0]
    if (!file) return

    setUploading(true)
    const ext = file.name.split('.').pop()
    const path = `${recordId}/${Date.now()}.${ext}`

    const { data, error } = await supabase.storage
      .from(bucket)
      .upload(path, file, { cacheControl: '3600', upsert: false })

    if (error) {
      alert('Error al subir imagen: ' + error.message)
      setUploading(false)
      return
    }
  }
}
```

```
const { data: { publicURL } } = supabase.storage
  .from(bucket)
  .getPublicUrl(data.path)

onUpload(publicURL)
setUploading(false)
}

return (
  <div>
    <input type="file" accept="image/*" onChange={handleUpload} disabled={uploading} />
    {uploading && <p>Subiendo...</p>}
  </div>
)
```

15. Seguridad y privacidad

15.1 Capas de seguridad

El sitio aplica defensa en profundidad con cuatro capas:

1. **DB (RLS)**: la última línea. Aunque el frontend falle, RLS bloquea accesos no autorizados.
2. **Server Actions (auth check)**: verifican `auth.uid()` + `is_admin()` antes de cualquier mutación.
3. **Middleware (route guard)**: redirige `/admin/*` a `/admin/login` si no hay sesión válida.
4. **Frontend (UX)**: oculta botones y links según rol, pero nunca confía en esto como seguridad.

15.2 Manejo de secretos

Reglas absolutas:

- Secretos nunca en código. Siempre en variables de entorno.
- Service role key de Supabase **solo en código server**. Imports en componentes con `'use client'` deben fallar el build.
- `.env.local` en `.gitignore`. `.env.example` documentado sin valores reales.
- Rotar keys si se exponen accidentalmente. Supabase y Stripe permiten rotación.

Detección automática:

GitHub Secret Scanning está activo por default. Si un secreto se commitea por accidente, GitHub lo detecta y envía alerta.

Setup de Pre-commit hook (recomendado):

```
# .husky/pre-commit
pnpm lint-staged
```

Con lint-staged configurado para detectar patrones de secretos.

15.3 RLS: revisión exhaustiva

Cada tabla tiene políticas explícitas. Verificación obligatoria antes de merge a main:

```
-- Verificar que TODAS las tablas tienen RLS habilitada

select schemaname, tablename, rowsecurity
from pg_tables
where schemaname = 'public'
and rowsecurity = false;

-- No debe retornar nada
```

Test manual de RLS desde Supabase Studio:

1. Setear `auth.uid()` a un user que NO es admin.
2. Intentar `select`, `insert`, `update`, `delete` en cada tabla.
3. Verificar que solo funcionan las operaciones permitidas.

15.4 Validación de inputs

Todo input externo se valida con Zod:

- Formularios (react-hook-form con zodResolver)
- Server Actions (safeParse antes de mutar)
- Webhooks (parsear el body recibido)
- URL params y searchParams

Nunca confiar en validación solo del frontend. La validación se duplica en server.

15.5 Protección contra ataques comunes

XSS:

- React escapa por default. Nunca usar `dangerouslySetInnerHTML` con input de usuario.
- Para rich text de posts, sanitizar con `isomorphic-dompurify` antes de renderizar.

CSRF:

- Server Actions de Next.js incluyen protección CSRF nativa (verifican origen).

SQL Injection:

- Supabase SDK usa prepared statements. Nunca construir SQL como string.
- Si se usa raw SQL, usar parámetros vinculados.

Webhook spoofing:

- Stripe: verificar firma con `stripe.webhooks.constructEvent`.
- PayPal: verificar evento con su API antes de procesar.

Brute force en login:

- Supabase Auth tiene rate limiting nativo en magic link.
- Adicional: Vercel WAF (en plan Pro, no MVP).

15.6 Privacidad de datos personales

Datos personales que el sitio captura:

Dato	Origen	Tratamiento
Email de admin	Login	Solo en <code>auth.users</code> , no expuesto
Email de voluntario	Form aplicación	Solo en <code>volunteers</code> , no en lecturas públicas
Phone de voluntario	Form aplicación	Idem
Email de donante	Form donación	Solo admin lo ve
Nombre de donante	Form donación	Público SOLO si opt-in

Privacy policy mínima (post-MVP): redactar política simple en `/privacidad` explicando qué datos se capturan, para qué, y cómo se borran a solicitud.

Derecho de borrado: admin puede eliminar manualmente cualquier registro de `volunteers` o `donations` desde el panel.

15.7 Logs y auditoría

Mínimo para MVP:

- Logs de Vercel para requests y errores.
- Sentry para errores no manejados.
- Logs de Supabase para queries SQL.

Post-MVP recomendado:

- Tabla `audit_log` que registra cada acción admin (insert, update, delete).

15.8 Backups

Supabase incluye backups automáticos:

- Free tier: 7 días de backups Point-In-Time Recovery.
- Pro tier: 30 días.

Adicional para MVP: semanalmente, hacer un dump manual de tablas críticas:

```
pg_dump $DATABASE_URL --schema=public > backup-$(date +%Y%m%d).sql
```

Guardar en almacenamiento externo (Google Drive del admin, por ejemplo).

16. SEO e internacionalización

16.1 Metadata por página

Cada página implementa `generateMetadata` con título, descripción, OG image y locale alternates:

```
export async function generateMetadata({ params }: PageProps): Promise<Metadata> {
  const { locale } = await params
  const t = await getTranslations({ locale, namespace: 'meta.home' })

  return {
    title: t('title'),
    description: t('description'),
    openGraph: {
      title: t('title'),
      description: t('description'),
      url: `${process.env.NEXT_PUBLIC_SITE_URL}/${locale}`,
      siteName: 'Bomberos Todos Santos',
      locale: locale === 'es' ? 'es_MX' : 'en_US',
      images: [{ url: '/og-default.jpg', width: 1200, height: 630 }],
      type: 'website',
    },
    twitter: { card: 'summary_large_image' },
  }
}
```

```
alternates: {
  canonical: `${process.env.NEXT_PUBLIC_SITE_URL}/${locale}`,
  languages: {
    'es-MX': `${process.env.NEXT_PUBLIC_SITE_URL}/es`,
    'en-US': `${process.env.NEXT_PUBLIC_SITE_URL}/en`,
  },
},
}
```

16.2 Sitemap dinámico

src/app/sitemap.ts:

```
import type { MetadataRoute } from 'next'
import { createClient } from '@lib/supabase/server'

export default async function sitemap(): Promise<MetadataRoute.Sitemap> {
  const baseUrl = process.env.NEXT_PUBLIC_SITE_URL!
  const supabase = await createClient()

  const [{ data: needs }, { data: events }, { data: posts }] = await Promise.all([
    supabase.from('equipment_needs').select('slug, updated_at').eq('published', true),
    supabase.from('events').select('slug, updated_at').eq('published', true),
    supabase.from('posts').select('slug, updated_at').eq('published', true),
  ])

  const locales = ['es', 'en']

  const staticPaths = ['', '/necesidades', '/donar', '/transparencia', '/eventos',
    '/voluntarios', '/nosotros', '/noticias']

  const staticEntries = locales.flatMap((locale) =>
    staticPaths.map((path) => ({
      url: `${baseUrl}/${locale}${path}`,
      lastModified: new Date(),
      changeFrequency: 'weekly' as const,
      priority: path === '' ? 1.0 : 0.8,
    })))
```

```

)

const needEntries = (needs ?? []).flatMap((n) =>
  locales.map((locale) => ({
    url: `${baseUrl}/${locale}/necesidades/${n.slug}`,
    lastModified: new Date(n.updated_at),
    changeFrequency: 'weekly' as const,
    priority: 0.7,
  })))
)

// similar para events y posts

return [...staticEntries, ...needEntries /* , ...eventEntries, ...postEntries */]
}

```

16.3 Robots.txt

src/app/robots.ts:

```

import type { MetadataRoute } from 'next'

export default function robots(): MetadataRoute.Robots {
  return {
    rules: [
      { userAgent: '*', allow: '/', disallow: ['/admin/', '/api/'] },
    ],
    sitemap: `${process.env.NEXT_PUBLIC_SITE_URL}/sitemap.xml`,
  }
}

```

16.4 Open Graph images dinámicas

src/app/api/og/[type]/route.tsx genera imágenes 1200x630 al vuelo usando next/og (basado en Satori):

- /api/og/need?id=xxx → genera OG image con título de necesidad + costo + foto.
- /api/og/event?id=xxx → genera OG image con título + fecha + lugar.
- /api/og/post?id=xxx → genera OG image con título + cover.

16.5 Structured data (JSON-LD)

Cada página relevante incluye Schema.org JSON-LD:

Home y nosotros: NGO + LocalBusiness:

```
<script
  type="application/ld+json"
  dangerouslySetInnerHTML={{
    __html: JSON.stringify({
      '@context': 'https://schema.org',
      '@type': 'NGO',
      name: 'Bomberos de Todos Santos',
      url: process.env.NEXT_PUBLIC_SITE_URL,
      logo: `${process.env.NEXT_PUBLIC_SITE_URL}/logo.png`,
      address: { /* ... */ },
      sameAs: [/* facebook, instagram */],
    }),
  }}
/>
```

Eventos: Event schema.

Noticias: NewsArticle schema.

Donaciones: DonateAction o MonetaryGrant.

16.6 Configuración de next-intl

src/lib/i18n/routing.ts:

```
import { defineRouting } from 'next-intl/routing'
import { createNavigation } from 'next-intl/navigation'

export const routing = defineRouting({
  locales: ['es', 'en'],
  defaultLocale: 'es',
  localePrefix: 'always', // siempre /es/* o /en/*
  localeDetection: true, // detecta del Accept-Language
})
```



```
export const { Link, redirect, usePathname, useRouter } = createNavigation(routing)
```

src/lib/i18n/request.ts:

```
import { getRequestConfig } from 'next-intl/server'
import { routing } from './routing'

export default getRequestConfig(async ({ requestLocale }) => {
  let locale = await requestLocale

  if (!locale || !routing.locales.includes(locale as any)) {
    locale = routing.defaultLocale
  }

  return {
    locale,
    messages: (await import(`./${locale}.json`)).default,
  }
})
```

16.7 Estructura de archivos i18n

Ejemplo de src/lib/i18n/es.json (parcial):

```
{
  "common": {
    "donate": "Donar",
    "learn_more": "Conocer más",
    "loading": "Cargando...",
    "error": "Error"
  },
  "meta": {
    "home": {
      "title": "Bomberos de Todos Santos – Protegemos a nuestra comunidad",
      "description": "Bomberos voluntarios que sirven a Todos Santos, BCS. Conoce nuestras necesidades de equipo y cómo ayudar."
    }
  },
  "nav": {
```

```
"needs": "Necesidades",
"transparency": "Transparencia",
"events": "Eventos",
"about": "Nosotros",
"volunteers": "Voluntarios",
"courses": "Cursos",
"news": "Noticias"
},
"home": {
"hero": {
"badge": "Necesitamos tu ayuda",
"headline": "Equipo que salva vidas en Todos Santos",
"description": "Somos voluntarios. Nuestro equipo lo dona gente como tú. Cada pieza tiene un nombre, un costo y una historia.",
"cta_primary": "Ver necesidades urgentes",
"cta_secondary": "Donar ahora"
},
"stats": {
"calls": "Llamadas atendidas",
"volunteers": "Voluntarios activos",
"donations": "Donaciones",
"years": "Años sirviendo"
}
},
"needs": {
"title": "Equipo que necesitamos",
"subtitle": "Cada pieza específica, con su costo real y estado actual",
"priority": {
"critical": "Crítica",
"high": "Alta",
"medium": "Media",
"low": "Baja"
}
},
"category": {
"epp": "Equipo de protección",
```

```
"tools": "Herramientas de rescate",
"vehicles": "Vehículos",
"medical": "Equipo médico",
"communications": "Comunicación",
"station": "Estación",
"other": "Otros"
},
"status": {
  "open": "Abierto",
  "partial": "Parcial",
  "fulfilled": "Cubierto"
},
"progress": "{received} de {needed} cubiertos",
"donate_this": "Donar este equipo",
"donate_money": "Donar dinero",
"empty": "No hay necesidades abiertas en este momento"
}
}
```

17. Plan de QA y lanzamiento

17.1 Checklist pre-lanzamiento (día 5-6)

Funcionalidad:

- ☐ Crear, editar, borrar necesidad funciona
- ☐ Subir imágenes a Storage funciona
- ☐ Verificar que las imágenes se sirven correctamente (CDN)
- ☐ Filtros en /necesidades funcionan con URL params
- ☐ Detalle de necesidad muestra galería navegable
- ☐ Formulario de donación intent envía email a admin
- ☐ PayPal button funciona en sandbox
- ☐ Stripe está OFF (verificar en site_settings)
- ☐ Voluntario submitea, recibe email, admin lo ve, aprueba, recibe email
- ☐ CRUD de eventos funciona

- ☐ CRUD de posts funciona
- ☐ Home muestra featured needs, próximos eventos, últimas noticias

Bilingüe:

- ☐ Toggle ES/EN cambia idioma sin perder página
- ☐ URLs reflejan idioma (/es/... y /en/...)
- ☐ Todo el UI traducido (revisar key por key)
- ☐ Contenido dinámico (needs, events, posts) muestra título según locale
- ☐ Fallback a español si no hay traducción

Responsive:

- ☐ Mobile 375px: nav hamburger, layouts apilados, fuentes legibles
- ☐ Tablet 768px: grid de 2 columnas donde aplica
- ☐ Desktop 1280px+: layouts completos, max-w aplicado

Accesibilidad:

- ☐ Lighthouse Accessibility ≥ 95 en home, /necesidades, /donar
- ☐ Navegación por teclado funciona en formularios
- ☐ Skip nav link funciona
- ☐ Contraste verificado en todas las páginas
- ☐ Alt en todas las imágenes

Performance:

- ☐ Lighthouse Performance ≥ 90 en home
- ☐ LCP $< 2.5s$
- ☐ CLS < 0.1
- ☐ Imágenes en next/image con sizes correctos
- ☐ Fuentes con font-display swap

SEO:

- ☐ Lighthouse SEO ≥ 95
- ☐ Metadata en cada página
- ☐ sitemap.xml accesible y completo
- ☐ robots.txt correcto
- ☐ OG images se generan correctamente
- ☐ Canonical URLs sin duplicados

Seguridad:

- ☐ /admin/* redirige a /admin/login si no autenticado
- ☐ Usuario no-admin no puede acceder a /admin/* aunque tenga sesión

- [] RLS bloquea lecturas/escrituras no autorizadas
- [] Secretos no expuestos en bundle de cliente (revisar Network tab)
- [] Webhooks verifican firma

Errores:

- [] error.tsx muestra mensaje user-friendly
- [] not-found.tsx funciona para slugs inexistentes
- [] Sentry recibe errores de prueba
- [] No console.errors en consola del browser

17.2 Smoke test en producción

Tras merge a main y deploy:

1. Abrir [https://\[dominio\]](https://[dominio]) → home carga
2. Click "Donar ahora" → /donar carga
3. Click toggle EN → cambia idioma
4. Click necesidad destacada → detalle carga
5. Llenar form de voluntario → submit funciona
6. Login admin → magic link llega al email
7. Crear necesidad de prueba → aparece en pública
8. Verificar emails llegando a RESEND_FROM_EMAIL
9. Lighthouse en producción ≥ 90 perf, ≥ 95 todo lo demás

17.3 Plan de difusión inicial

Día de lanzamiento:

- Post en Facebook de los bomberos con link al sitio.
- Mensaje en WhatsApp grupos locales.
- Email a donantes históricos (si existen direcciones).
- Mención a contactos en Todos Santos (cámara de comercio, expat community).

Primera semana:

- Subir 3-5 noticias iniciales contando historia de la central.
- Pedir a 2-3 voluntarios actuales que llenen perfiles públicos en /nosotros.
- Contactar prensa local (Diario El Independiente, etc.).

17.4 Plan de capacitación al admin

Sesión 1 (día 6, 1 hora):

- Login + tour del panel.
- Crear una necesidad de prueba paso a paso.
- Verificar una donación de prueba.
- Aprobar un voluntario de prueba.
- Publicar una noticia simple.

Material de soporte:

- ADMIN_GUIDE.md con screenshots.
- 3 Loom videos:
 - "Cómo subir una nueva necesidad de equipo" (8 min)
 - "Cómo verificar y publicar una donación" (5 min)
 - "Cómo aprobar voluntarios y crear eventos" (10 min)
- WhatsApp directo para preguntas las primeras 2 semanas.

18. Mantenimiento post-lanzamiento

18.1 Cadencia mínima recomendada

Frecuencia	Tarea
Semanal	Verificar donaciones pendientes, aprobarlas
Semanal	Revisar aplicaciones de voluntarios
Quincenal	Actualizar progreso de necesidades (quantity_received)
Mensual	Publicar una noticia o agradecimiento
Mensual	Revisar Vercel Analytics y Sentry
Trimestral	Actualizar fotos del equipo, métricas anuales
Anual	Reporte de transparencia descargable

18.2 Roadmap post-MVP

v1.1 (2-4 semanas después):

- Activar Stripe (cuando se confirme figura legal)
- Emisión automática de CFDI (si hay donataria autorizada)
- Newsletter con Mailchimp o Resend Audiences
- Registro a eventos con cupo limitado

v1.2 (1-3 meses después):

- Reporte anual descargable como PDF
- Galería pública del equipo recibido (antes/después)
- Mapa de incidentes anónimo (heatmap)
- Sección "Conoce a los bomberos" con perfiles individuales

v1.3 (3-6 meses):

- Tienda con merchandise (gorras, calendarios) para recaudar
- Programa de donantes recurrentes
- Dashboard público con métricas en tiempo real
- App móvil opcional con notificaciones

18.3 Costos operativos esperados año 1

Categoría	Mensual	Anual
Vercel Hobby	\$0	\$0
Supabase Free	\$0	\$0
Resend Free	\$0	\$0
Sentry Free	\$0	\$0
Plausible (opcional)	\$9	\$108
Dominio	—	\$15-25
Comisiones de pago (estimado)	—	3-4% de donaciones recibidas

Total operativo: \$25-135 USD/año (sin contar comisiones).

Si el tráfico crece y se excede Supabase free tier, upgrade a Pro: \$25/mes.

18.4 Documentación a mantener

Documento	Propietario	Actualización
README.md	Desarrollador	Cuando cambia setup
ADMIN_GUIDE.md	Desarrollador + admin	Cuando se agrega feature visible
DEPLOYMENT.md	Desarrollador	Cuando cambia hosting o config
WORKFLOW.md	Desarrollador	Estable, raras actualizaciones
DECISIONS.md (ADRs)	Desarrollador	Cada decisión arquitectónica nueva

19. Riesgos y mitigaciones

19.1 Matriz de riesgos

Riesgo	Probabilidad	Impacto	Mitigación
Figura legal no se aclara → Stripe no activable	Alta	Medio	MVP funciona sin Stripe (PayPal + transferencia)
Sin fotos reales de calidad	Alta	Alto	Conseguirlas en día 0 o 1; tener placeholders dignos
Admin no mantiene contenido	Media	Alto	Plan de capacitación + WhatsApp de soporte + cadencia mínima
Donaciones bajas iniciales	Alta	Bajo	Definir éxito como "site funcional", no como métrica de donación inicial
Conflictos de merge entre agentes	Media	Medio	Zonas calientes claramente definidas; rebase frecuente
Bug crítico en producción	Baja	Alto	Rollback rápido en Vercel; Sentry detecta automáticamente
Excede free tier de Supabase	Baja	Bajo	Métrica de uso visible; upgrade es \$25/mes
Spam en form de voluntarios	Media	Bajo	Rate limit + honeypot field; reCAPTCHA si crece
Hack o leak de datos	Baja	Alto	RLS estricta; secretos solo en env vars; auditoría inicial
Vercel/Supabase outage	Baja	Alto	Aceptable para MVP; documentación de migración a alternativas
Pérdida de admin (renuncia)	Media	Alto	Más de un admin con acceso; documentación clara

19.2 Plan de contingencia

Si el sprint se atrasa:

- Día 5: si no se llega a integración, recortar features no críticas (posts puede ser solo lectura desde admin de Supabase, sin UI custom).

- Día 6: si no llega al go-live, hacer beta privada con URL no anunciada y completar la siguiente semana.

Si un agente bloquea al otro:

- El humano interviene y completa la zona caliente.
- Reasignar feature al otro agente si el primero se atora.

Si la figura legal demora mucho:

- Lanzar sin Stripe.
- Mantener Stripe como flag activable cuando llegue.
- No demorar el lanzamiento del sitio entero por esto.

20. Apéndices

20.1 Apéndice A: schema SQL completo

El archivo supabase/migrations/00001_initial_schema.sql contiene el schema completo. Se entrega como archivo separado schema.sql. Contiene:

- Extensiones: uuid-oss, pg_trgm
- 6 enums: equipment_category, priority_level, need_status, event_type, donation_type, post_type, volunteer_status
- 8 tablas: equipment_needs, donations, events, posts, volunteers, station_stats, site_settings, admin_users
- 1 vista: v_equipment_needs_with_progress
- 1 función: is_admin() con security definer
- 18 políticas RLS
- 12 índices
- 8 triggers de updated_at

20.2 Apéndice B: archivos de configuración

.env.example:

```
# Supabase
NEXT_PUBLIC_SUPABASE_URL=https://xxx.supabase.co
NEXT_PUBLIC_SUPABASE_ANON_KEY=your-anon-key
SUPABASE_SERVICE_ROLE_KEY=your-service-role-key
```

```
# Site

NEXT_PUBLIC_SITE_URL=http://localhost:3000

# Stripe (preparado, off por default)

STRIPE_SECRET_KEY=sk_test...

STRIPE_WEBHOOK_SECRET=whsec...

NEXT_PUBLIC_STRIPE_PUBLISHABLE_KEY=pk_test...

# PayPal

PAYPAL_CLIENT_ID=...

PAYPAL_CLIENT_SECRET=...

NEXT_PUBLIC_PAYPAL_CLIENT_ID=...

# Resend

RESEND_API_KEY=re...

RESEND_FROM_EMAIL=no-reply@example.com

# Sentry

NEXT_PUBLIC_SENTRY_DSN=https://...

SENTRY_AUTH_TOKEN=...

# Analytics (opcional)

NEXT_PUBLIC_PLAUSIBLE_DOMAIN=example.com
```

.gitignore:

```
# dependencies

node_modules

.pnp

.pnp.js

# next.js

.next/

out/

build

# production

*.tsbuildinfo

next-env.d.ts
```

```
# misc

.DS_Store

*.pem

.env*.local

.env

# vercel

.vercel

# testing

coverage

.nyc_output

# IDE

.vscode/

.idea/

*.swp

*.sw0

# supabase

supabase/.branches

supabase/.temp
```

20.3 Apéndice C: comandos útiles del día a día

```
# Setup inicial

pnpm install

pnpm db:types # Generar tipos de Supabase

pnpm dev # Dev server en localhost:3000

# Día a día

pnpm dev

pnpm lint

pnpm typecheck

pnpm build

# Git

git checkout develop && git pull
```

```
git checkout -b feat/mi-feature

git add . && git commit -m "feat(scope): descripción"

git push -u origin feat/mi-feature

# Abrir PR en GitHub

# Mantener rama al día

git fetch origin

git rebase origin/develop

# Si hay conflictos en rebase

# Resolver conflictos en archivos

git add <archivos-resueltos>

git rebase --continue

# O cancelar:

git rebase --abort

# Supabase local (opcional, post-MVP)

supabase start

supabase db reset

supabase db diff

supabase gen types typescript --local > src/types/database.ts

# Vercel CLI (opcional)

npx vercel link

npx vercel env pull .env.local

npx vercel logs
```

20.4 Apéndice D: troubleshooting común

"Build falla en Vercel pero local funciona":

- Variables de entorno no configuradas en Vercel.
- Versión de Node distinta (verificar engines en package.json).
- Dependencias del lockfile desactualizadas.

"RLS bloquea consultas que deberían funcionar":

- Verificar que `auth.uid()` no es null en la sesión.
- Probar la query directamente en Supabase SQL Editor con `set role authenticated`.
- Revisar que el usuario está en `admin_users`.

"¡18n no carga textos":

- Verificar messages configurado en root layout con `NextIntlClientProvider`.
- Confirmar que `setRequestLocale(locale)` se llama en Server Components.
- Verificar que la key existe en ambos `es.json` y `en.json`.

"Imágenes no cargan":

- `next.config.mjs` debe tener el dominio de Supabase Storage en `images.remotePatterns`.
- Verificar que el bucket es público.
- Confirmar que las políticas de Storage permiten lectura.

"Server Action no se ejecuta":

- Verificar que el archivo empieza con `'use server'` en la línea 1.
- Verificar que se exporta como `async function`.
- Confirmar que se llama desde un client component o desde un form.

"Type errors después de cambio de schema":

- Regenerar tipos: `pnpm db:types`.
- Verificar que `database.ts` está actualizado.
- Reiniciar TypeScript server en el editor.

20.5 Apéndice E: glosario

Término	Significado
Server Action	Función async marcada con <code>'use server'</code> que corre en el servidor pero se invoca desde el cliente como si fuera local
Route Handler	Endpoint HTTP tradicional en App Router, en <code>app/api/*/route.ts</code>
Server Component	Componente React que renderiza en el servidor, puede hacer fetch async directo
Client Component	Componente con <code>'use client'</code> , corre en el browser, puede usar state y efectos
RLS	Row Level Security, políticas de Postgres que filtran filas según el usuario
BaaS	Backend as a Service (Supabase, Firebase)
CFDI	Comprobante Fiscal Digital por Internet, factura electrónica mexicana
Donataria autorizada	A.C. con permiso SAT para emitir CFDI de donaciones deducibles

Término	Significado
Magic link	Login sin password, mediante link enviado al email
OG image	Imagen para Open Graph protocol, lo que se muestra al compartir el link en redes
Conventional Commits	Estándar de mensajes de commit (feat:, fix:, etc.)
Preview deploy	Despliegue automático de Vercel para cada PR con URL única
Edge function	Función serverless ejecutada en CDN edge (Supabase o Vercel)

20.6 Apéndice F: referencias y documentación externa

- **Next.js 15:** <https://nextjs.org/docs>
- **Supabase:** <https://supabase.com/docs>
- **next-intl:** <https://next-intl-docs.vercel.app>
- **shadcn/ui:** <https://ui.shadcn.com>
- **Tailwind CSS:** <https://tailwindcss.com/docs>
- **Zod:** <https://zod.dev>
- **react-hook-form:** <https://react-hook-form.com>
- **Stripe Connect:** <https://stripe.com/docs/connect>
- **PayPal SDK:** <https://developer.paypal.com/docs/checkout/>
- **Resend:** <https://resend.com/docs>
- **Vercel deployment:** <https://vercel.com/docs>

Cierre

Este documento es la fuente única de verdad para el proyecto durante el sprint y el primer año de operación. Cualquier desviación se documenta como un Architecture Decision Record en `docs/DECISIONS.md`.

Próximos pasos inmediatos:

1. Confirmar figura legal de los bomberos para decidir activación de Stripe.
2. Recolectar fotografía real del equipo y la central (día 0).
3. Setup del repo y Vercel siguiendo sección 12.1.
4. Lanzar día 0 con scaffold completo.
5. Iniciar día 1 con briefings a ambos agentes según sección 13.

Si algo en este documento no está claro o necesita revisión, registrar como issue en GitHub con label `documentation` antes de seguir.

Documento mantenido por: el equipo del proyecto

Última actualización: mayo 2026